
Deep Orthogonal Decomposition for surrogate modelling of time dependent PDEs

written and submitted by DAWID KOTOWSKI

A thesis presented for the degree of
Master of Science

Supervisor: PROF. DR. MARIO OHLBERGER
Faculty of Mathematics and Computer Science
University Münster
Germany, DE
submitted on August 24, 2025

Contents

0.1	Notation	1
0.1.1	Partial Differential Equations	1
0.1.2	Neural Networks	1
1	Introduction	3
1.1	Time Dependent PDEs	3
1.2	Neural networks	4
1.3	Motivation	4
1.4	Literature review	4
1.5	Outline	4
2	Theoretical Background	5
2.1	Linear Algebra	5
2.2	Parametric Partial Differential Equations	5
2.2.1	Sobolev Spaces	5
2.2.2	Existence and Uniqueness	7
2.2.3	Reduced Basis Framework	8
2.3	Neural Networks	10
2.3.1	Network concatenations	12
2.3.2	Network parallelization	13
2.3.3	Standard Neural Networks	14
2.3.4	Approximation Theory	14
3	Formulation of the General Problem	25
3.1	Time-dependent parametric PDE	25
3.2	Projection-based Reduced Order Models	28
3.2.1	Galerkin Proper Orthogonal Decomposition	28
3.2.2	Greedy Proper Orthogonal Decomposition	29
3.3	Data-driven Reduced Order Models	30
3.3.1	Deep Orthogonal Decomposition based Reduction	31
3.3.2	DOD+DNN	32
3.3.3	DOD-DL-ROM	32
3.3.4	CoLoRA-DL-ROM	32
4	Approximation Theory for Reduced Order Models	33
4.1	Relative Error of POD+DNNs	33
4.2	Relative Error of DOD+DNNs	35
4.3	Complexity Bounds for POD-DL-ROMs	39
4.4	Complexity Bounds for DOD-DL-ROMs	42

4.5	Comparison between existing Techniques	43
5	Numerical Experiments	45
6	Conclusion	47

Todo list

Write an Introduction	3
add prove	5
doesnt work	7
Give a reasonable weak formulation framework to appreciate gram matrix	8
Introduce the general notion of the POD	8
Introduce KnW and its relation to the continous case	8
include theorem 6.1 from quarteroni2015rb	9
Motivate this with some example.	11
maybe include proof?	13
maybe include proof?	14
Why?	15
argue this	20
Describe basic already existing techniques via literature	25
Introduce Proper Orthogonal Decomposition	25
Introduce Autoencoders	25
Introduce Deep Orthogonal Decomposition	25
Definitely add traditional convergence and computational cost.	25
Rework with regard to gram matrix	29
Introduce POD-DL-ROM	30
Introduce both DOD-DL-ROM and DOD+DNN	31
Introduce CoLoRA DL-ROM	32
Include proof and discussion for general DL-ROMs	33
Discuss relative errors for the plain POD+DNN	33
Proof this	35
Extrapolate to DOD+DNN	35
Rework the proof and statement according to the setting	35
Do the Yarotski argument with the appropriate number of layers and weights.	39
Include discussed results with proof of Brivio et al.	39
Make this rigorous	41
Make this rigorous	41
Make this rigorous	41
Extrapolate to DOD-DL-ROM	42
Include the cost via Yarotski	43
Argue the actual part for the NN error	43
Include all numerical experiments	45
Find increase of performance depending on latent dimension	45
Give benchmark on pass forward times	45
Discuss comparison between different DL-ROMs	45

Discuss results	47
---------------------------	----

Insert outlook for future ideas and implementation	47
--	----

0.1 Notation

We set $\mathbb{N} := \{1, 2, \dots\}$, $\mathbb{N}_0 := \{0, 1, \dots\}$ and $\mathbb{N}_{\geq n} := \{n, n+1, \dots\}$ for $n \in \mathbb{N}$. Next the set $\mathbb{R}^+ := \{x \in \mathbb{R} \mid x \geq 0\}$ will define all real numbers, which are non-negative. Furthermore for any set A , we will write $|A|$ to denote its cardinality. If $x \in \mathbb{R}$, the gaussian ceiling function is given via $\lceil x \rceil := \min\{k \in \mathbb{Z} : k \geq x\}$.

Further if $d \in \mathbb{N}$ and $\|\cdot\|$ is a norm, then for $x \in \mathbb{R}^d$ and $r > 0$ we set $B_{r, \|\cdot\|}(x)$ as the open ball around x with radius r . Additionally $|\cdot|$ denotes the regular euclidean norm on $\mathbb{R}^d$. We usually do not differ between vectors and scalars in ubiquitous notation.

The transpose of a matrix $A \in \mathbb{R}^{d_1 \times d_2}$ for some $d_1, d_2 \in \mathbb{N}$ is given by $A^T \in \mathbb{R}^{d_2 \times d_1}$.

Let $(a_n)_{n \in \mathbb{N}} \in \mathbb{R}$ be some sequence, then we say $(a_n)_n$ has (*convergence of*) *order* n , if there is $c \in \mathbb{R}$ independent of n , s.t. $a_n = c \cdot n$ for all $n \in \mathbb{N}$. We then denote $a_n = \mathcal{O}(n)$.

When writing $a \ll b$ for $a, b \in \mathbb{R}$, the reader can infer, that $a < b$, but also, that the different is "substantial". We do not provide a rigorous definition of this, even though some literature does on occasion.

Moreover, we define the *multiindex* notation as follows: For $\alpha = (\alpha_1, \dots, \alpha_d) \in \mathbb{N}_0^d$, we write $|\alpha| := \alpha_1 + \dots + \alpha_d$ and $\alpha! := \alpha_1! \dots \alpha_d!$. If $x \in \mathbb{R}^d$, then we set

$$x^\alpha := \prod_{i=1}^d x_i^{\alpha_i}.$$

0.1.1 Partial Differential Equations

For a function $f : X \rightarrow Y$ we denote $\text{supp}(f) := \{x \in X \mid f(x) \neq 0\}$ as the *support* of f . Let $g : Y \rightarrow Z$ be another function, then $g \circ f : X \rightarrow Z$ is their *composition*. Setting the standard topology of $\mathbb{R}^d$ lets us define $\bar{A}$ for some $A \subset \mathbb{R}^d$ as the *closure* of A , and ∂A its *boundary*. If A is non-empty, we set $\text{diam}(A) := \sup_{x, y \in A} |x - y|$ as the diameter of A .

Let $\Omega \subset \mathbb{R}^d$ be open. For $n \in \mathbb{N}_0 \cup \{\infty\}$, we denote by $C^n(\Omega)$ the set of n -times continuously differentiable functions on Ω .

For $f \in C^n(\Omega)$ and $\alpha \in \mathbb{N}_0^d$ with $|\alpha| \leq n$ we write

$$D^\alpha f := \frac{\partial^{|\alpha|} f}{\partial x_1^{\alpha_1} \partial x_2^{\alpha_2} \dots \partial x_d^{\alpha_d}}.$$

A function $f : \Omega \rightarrow \mathbb{R}^m$ for $\Omega \subset \mathbb{R}^d$ is called *Lipschitz continuous*, if there is a constant $L \geq 0$ such that

$$|f(x) - f(y)| \leq L|x - y|.$$

We further denote with

$$L^p(\Omega) := \left\{ f : \Omega \rightarrow \mathbb{R} \mid f \text{ is measurable, and } \int_{\Omega} |f(x)|^p d\mu(x) < \infty \right\}$$

the Lebesgue space and L_{loc}^p as the space of $L^p(\Omega)$ functions with local support. If not specified otherwise we write $\|\cdot\|_p := \|\cdot\|_{L^p(\Omega)}$ for all $1 \leq p \leq \infty$. Note that in critical situations we tend to be explicit about the norms.

0.1.2 Neural Networks

Consider $d_1, d_2 \in \mathbb{N}$ and a matrix $A \in \mathbb{R}^{d_1 \times d_2}$, then the number of non-zero entries of A is given by

$$\|A\|_{\ell^0} := |\{(i, j) : A_{i,j} \neq 0\}|.$$

Let $B \in \mathbb{R}^{d_1 \times d_3}$ be some other matrix for $d_3 \in \mathbb{N}$. Then we write

$$\begin{bmatrix} A & B \end{bmatrix} \in \mathbb{R}^{d_1 \times (d_2 + d_3)} \quad \text{or} \quad \begin{bmatrix} A & | & B \end{bmatrix} \in \mathbb{R}^{d_1 \times (d_2 + d_3)},$$

for the stacking of matrices, where the latter notation is meant to symbolize a strong delineation between blocks. Similar applies to vertical stacking, if $A \in \mathbb{R}^{d_1 \times d_2}$ and $B \in \mathbb{R}^{d_3 \times d_2}$. As a special case, this applies to vectors as well.

Chapter 1

Introduction

Write an Introduction

1.1 Time Dependent PDEs

The study and analysis of partial differential equations (PDEs) began with the works of Euler, d'Alembert, Lagrange and Laplace (see (Brezis & Browder, 1998) for this introduction). As so often in mathematics it started off with a set of simple problems, such as the vibrating string in one dimension as modelled by d'Alembert (1752)

$$\frac{\partial^2}{\partial t^2}u(x, t) = \frac{\partial^2}{\partial x^2}u(x, t), \quad \text{for } u \in C^2(\mathbb{R} \times \mathbb{R}^+), x \in \mathbb{R} \text{ and } t \in \mathbb{R}^+,$$

the extension to three dimensions by Euler (1759) and Bernoulli (1762), the Laplace equation for gravitational fields (1780)

$$\Delta u(x) = 0, \quad \text{for } u \in C^2(\mathbb{R}^3) \text{ and } x \in \mathbb{R}^3,$$

and finally the heat equation, as introduced by Fourier (1810 - 1822)

$$\frac{\partial}{\partial t}u(x, t) = \Delta u(x, t), \quad \text{for } u \in C^2(\mathbb{R}^3 \times \mathbb{R}^+), x \in \mathbb{R}^3 \text{ and } t \in \mathbb{R}^+.$$

As any posed problem calls for a solution to it, there have been simultaneous developments with regard to the calculation methods; the method of separation of variables was given by Euler and d'Alembert for the study of spherical harmonics and Fourier introduced similar ideas for the heat equation. For the Laplace equation, the Green functions were introduced in 1835. Green also discovered an important principle for solvability of the (later so called) Dirichlet problem

$$\begin{cases} \Delta u(x) = 0, & \text{for } x \in \Omega \subset \mathbb{R}^3, \\ u(x) = g(x), & \text{for } x \in \partial\Omega, \end{cases}$$

for some function $g \in C^0(\mathbb{R}^3)$, which revolved around minimizing an energy functional $E : C^2(\mathbb{R}^3) \rightarrow \mathbb{R}^+$, thus founding the variational principle.

Though these were first attempts at solving such problems, the idea of Green in particular led to a few analytical problems, by the lack of mathematical rigor. Together with the contribution of Cauchy on the *initial value* problem, this laid the groundwork for the field of *Existence Theory*, which tries to establish existence results for PDEs.

Notable here are the alternating method of Schwarz (1870) using the idea of domain decomposition and especially Neumanns method of integral equations developed in 1877, which would go on to be the starting point for the weak form and subsequently the Sobolev spaces. These were a necessity after B. Levi showed in 1906, that a general minimizing sequence for the Dirichlet integral was a Cauchy sequence and would therefore converge in the right space, which was then given due to the Sobolev embedding theorems.

To tackle the problem arising from "exotic" examples and the complicated nature of some PDEs, Hadamard proposed a classification of the most common linear PDEs using its characteristic polynomial, which is given by replacing all partial derivatives $\partial/\partial x_j$ by the algebraic variable ξ_j . In that case the PDE is called

1. *elliptic*, if after a change of variables the polynomial can be written as $\xi_1^2 + \xi_2^2 + \dots + \xi_n^2$,
2. *parabolic*, if after a change of variables the polynomial can be written as $\xi_1 + \xi_2^2 + \dots + \xi_n^2$,
3. *hyperbolic*, if after a change of variables the polynomial can be written as $\xi_1^2 - (\xi_2^2 + \dots + \xi_n^2)$.

These analytical results can be expanded a lot more, but what is most interesting in terms of this paper, are the Sobolev spaces. They, and the closely connected Bochner spaces, are the relevant solution spaces to the limits of all, in some sense, consistent numerical schemes, since the solutions are at least embedded in them. This follows quite natural, as any numerical scheme is basically bounded by the necessity of being ran on a computer and thus having a finite dimensional space it defines its approximation on, and at the very least the Sobolev and Bochner spaces admit any piecewise smooth function.

1.2 Neural networks

1.3 Motivation

1.4 Literature review

1.5 Outline

Chapter 2

Theoretical Background

2.1 Linear Algebra

As preparation for the coming Bauer Fike Theorem, we want to define a matrix norm for the space $\mathbb{R}^{N \times N}$, where $N \in \mathbb{N}$, as follows

$$\|A\|_{\text{op}} := N \cdot \max_{1 \leq i, j \leq N} |[A]_{ij}|.$$

This satisfies the wanted property mentioned in the original paper (Bauer & Fike, 1960, p. 137-141); any operator norm such that

$$\|Ax\| \leq \|A\|_{\text{op}} \|x\|, \quad \text{for all } x \in \mathbb{R}^N$$

for some norm $\|\cdot\|$ on $\mathbb{R}^N$ can be used for the statement of the following theorem:

Theorem 2.1 (Bauer-Fike Theorem). *Let $K \in \mathbb{R}^{N \times N}$ be some diagonalizable matrices with σ_k eigenvalues and $V = (\xi_1, \dots, \xi_N) \in \mathbb{R}^{N \times N}$ be the stacked eigenvectors matrix of K and $\delta K \in \mathbb{R}^{N \times N}$ some symmetric matrix. Then if λ_k are the eigenvalues of $K + \delta K$, we have*

$$\min_k |\lambda_k - \sigma_k| \leq \kappa(V) \| (K + \delta K) - K \|_{\text{op}},$$

where $\kappa(V) = \|V\|_{\text{op}} \|V^{-1}\|_{\text{op}}$.

add prove

2.2 Parametric Partial Differential Equations

2.2.1 Sobolev Spaces

For our work with Partial Differential Equation, and subsequent error approximations, we want to introduce Sobolev spaces. These are generalized differentiable spaces, where a weak derivative is given, as an analogon to the classic derivative in the $C^n(\Omega)$ space. They are employed in order to exploit their compact embedding in the Hölder spaces, which are generalizations of $C^n(\Omega)$. We will not go in much detail about the theory of Sobolev spaces, since we mostly concern ourselves with finite dimensional subspaces, but some ground truth must be discussed.

For this subsection we primary use (Alt, 2016), and for the first upcoming definition (Dacorogna, 2004).

Definition 2.2 (Lipschitz Domain). Let $d \in \mathbb{N}$, and $\Omega \subset \mathbb{R}^d$. Then Ω is called a *Lipschitz domain* if for every $x \in \partial\Omega$ and some $r > 0$, there is $U := B_{r,|\cdot|}(x) \subset \mathbb{R}^d$ and an isomorphic map $H : Q \rightarrow B_{r,|\cdot|}(x)$ with $Q := \{x \in \mathbb{R}^d \mid \|x\|_{\ell^1} < d\}$ such that

$$H \in C^k(\overline{Q}), \quad H^{-1} \in C^k(\overline{U}), \quad H(Q_+) = U \cap \Omega, \quad H(Q_0) = U \cap \partial\Omega$$

where $Q_+ := \{x \in Q \mid x_d > 0\}$ and similarly $Q_0 := \{x \in Q \mid x_d = 0\}$, and H is only in $C^{0,1}$.

From now on, let $d \in \mathbb{N}$ and, if not given otherwise, $\Omega \subset \mathbb{R}^d$ be a Lipschitz domain.

Definition 2.3 (Weak Derivative). Let $\alpha \in \mathbb{N}_0^d$ be a multiindex. A function $u \in L_{\text{loc}}^1(\Omega)$ is said to have an α -th weak derivative, if there is a function $v_\alpha \in L_{\text{loc}}^1(\Omega)$ such that

$$\int_{\Omega} u D^\alpha \varphi = (-1)^{|\alpha|} \int_{\Omega} v_\alpha \varphi \quad \forall \varphi \in C_0^\infty(\Omega).$$

We then write $v_\alpha = D^\alpha u$.

Definition 2.4 (Sobolev Space). Let $n \in \mathbb{N}_0$ and $1 \leq p \leq \infty$. Then we define the *Sobolev space* as

$$W^{n,p}(\Omega) := \{f \in L^p(\Omega) \mid \|f\|_{W^{n,p}(\Omega)} < \infty\},$$

where the norm on this space is given via

$$\|f\|_{W^{n,p}(\Omega)} := \begin{cases} \left(\sum_{|\alpha| \leq n} \|D^\alpha f\|_{L^p(\Omega)}^p \right)^{1/p}, & \text{if } p < \infty \\ \max_{|\alpha| \leq n} \|D^\alpha f\|_{L^\infty(\Omega)}, & \text{if } p = \infty, \end{cases}$$

for any $f \in W^{n,p}(\Omega)$.

It is possible to extend this definition for $0 < n < 1$ to that of the *Sobolev-Slobodeckij spaces*. In the analysis of partial differential equations these are used to characterize the regularity of Sobolev functions, which are restricted to the boundary $\partial\Omega$. This restriction itself is called a trace operator. One of the statements of relevance for such fractal Sobolev spaces is the following:

If $\Omega \subset \mathbb{R}^d$ is a regular open set (for example, if Ω is a Lipschitz domain), and if $u \in W^{1,p}(\Omega)$, then in fact $u \mapsto u|_{\partial\Omega}$ as a trace operator is surjective from $W^{1,p}(\Omega)$ onto $W^{1-(1/p),p}(\partial\Omega)$ and

$$\|u|_{\partial\Omega}\|_{W^{1-(1/p),p}(\partial\Omega)} \leq C \|u\|_{W^{1,p}(\Omega)},$$

for some $C > 0$ independent of p and u , see (Brezis, 2011, p. 315). This of course only makes sense in light of the next definition.

Definition 2.5 (Sobolev-Slobodeckij Space). Let $0 < s < 1$ and $1 \leq p \leq \infty$. Then we define the *Sobolev-Slobodeckij space* as

$$W^{s,p}(\Omega) := \{f \in L^p(\Omega) \mid \|f\|_{W^{s,p}(\Omega)} < \infty\},$$

where the norm on this space is given via

$$\|f\|_{W^{s,p}(\Omega)} := \begin{cases} \left(\|f\|_{L^p(\Omega)}^p + \int_{\Omega} \int_{\Omega} \left(\frac{|f(x) - f(y)|}{|x - y|^{s+d/p}} \right)^p dx dy \right)^{1/p}, & \text{if } p < \infty \\ \max \left\{ \|f\|_{L^\infty(\Omega)}, \operatorname{ess\,sup}_{x,y \in \Omega} \frac{|f(x) - f(y)|}{|x - y|^s} \right\}, & \text{if } p = \infty, \end{cases}$$

for any $f \in W^{s,p}(\Omega)$.

Theorem 2.6 (Sobolev Embedding Theorem). *Let $m_1, m_2 \in \mathbb{N}_0$, $1 \leq p < \infty$ and $\alpha \in (0, 1)$. If we choose*

$$m_1 - \frac{d}{p} = m_2 + \alpha,$$

then the identity defines a continuous embedding

$$W^{m_1, p}(\Omega) \hookrightarrow C^{m_2, \alpha}(\overline{\Omega}).$$

If even $m_1 - \frac{d}{p} > m_2 + \alpha$, then the embedding is compact.

Proof. See (Alt, 2016, p. 10-13). □

2.2.2 Existence and Uniqueness

This subsection provides a brief introduction to one specific statement of existence and uniqueness, which we coincidentally can apply well to the projected version of our finite dimensional problem. Note, that of course, this does not justify any existence result for highly non-linear equations on a lot of upstream classical or even weak problems, like the famous incompressible Navier-Stokes Equation.

We start by defining the differential operator class, to which we can then apply the Hille-Yosida Theorem. Most notable, we know with that subclass, that any monotone operator, which acts on a finite dimensional space, is in that class, as will be shown afterwards.

Definition 2.7 (Maximal Monotone Operator). Let $A : D(A) \subset H \rightarrow H$ be an unbounded linear operator. It is *monotone*, if

$$\langle Av, v \rangle_H \geq 0, \quad \text{for all } v \in D(A),$$

and *maximal monotone*, if additionally for all $f \in H$ there is $u \in D(A)$ such that

$$u + Au = f.$$

Lemma 2.8. Let H be a finite dimensional real Hilbert space, and $A : H \rightarrow H$ a monotone operator according to Definition (2.7). Then A is maximal monotone.

Proof. By obvious identification with a basis, we know, that $H \cong \mathbb{R}^N$ for some $N \in \mathbb{N}$. Thus any element $u \in H$ can be expressed as

$$u = \sum_{j=1}^N u_j \varphi_j,$$

where $u_j \in \mathbb{R}$ and $\varphi_j \in H$ for all $j = 1, \dots, N$, and similarly for $i, j, k = 1, \dots, N$ there are $a_{ik} \in \mathbb{R}$ such that

$$Au = \sum_{i=1}^N \sum_{j=1}^N \sum_{k=1}^N a_{ik} u_j \langle \varphi_j, \varphi_i \rangle \varphi_k.$$

Thus the condition for A being maximal monotone simplifies to finding $[u_j]_{j=1}^N \in \mathbb{R}^N$ for an arbitrary $[f_j]_{j=1}^N \in \mathbb{R}^N$ where

doesn't work

$$u_j + \sum_{i=1}^N a_{ij} u_j = f_j$$

for all $j = 1, \dots, N$, which can obviously be done in general. $\square$

Theorem 2.9 (Hille-Yosida Theorem). *Let A be a maximal monotone operator, H some Hilbert space and $D(A) \subset H$. Then, given any $u_0 \in D(A)$ there is a unique function*

$$u \in C^1([0, \infty); H) \cap C([0, \infty); D(A))$$

satisfying

$$\begin{cases} \frac{\partial u}{\partial t} + Au = 0 & \text{on } [0, \infty), \\ u(0) = u_0 & \text{on } \{0\}. \end{cases}$$

Moreover,

$$\|u(t)\|_H \leq \|u_0\|_H,$$

for all $t \geq 0$.

Proof. See (Brezis, 2011, p. 185-191). $\square$

2.2.3 Reduced Basis Framework

Give a reasonable weak formulation framework to appreciate gram matrix

Introduce the general notion of the POD

Introduce KnW and its relation to the continous case

For this subsection we define $u(\mu_j, t_k) \in \mathbb{R}^{N_h}$ for $(\mu_j)_{j=1}^{N_s} =: \mathcal{P} \subset \mathbb{P}$ and $(t_k)_{k=1}^{N_t} =: \mathcal{T} \subset \mathbb{T}$ to be some vectors, later representing the solutions to a parametric PDE for a given parameter and time point in a given discrete subspace of the coefficient solution manifold. Here $\mathbb{P}$ represents an arbitrary parameter space, $\mathbb{T}$ some arbitrary time space, and $N_s, N_t \in \mathbb{N}$ be the parameter and time sample size respectively. It is assumed, that we sample the parameters and time points uniformly and iid from their respective spaces and that $\|u(\mu, t)\| < \infty$ for all $\mu \in \mathbb{P}$ and $t \in \mathbb{T}$. Let also

$$N_{\text{data}} := N_s \cdot N_t,$$

be the total sample size.

Here we will discuss prerequisites to the Proper Orthogonal Decomposition and related concepts, such as the Kolmogorov n -width, which lets us derive many key concepts and is an important quantity in the field of ROM methods. We base this subsection on (Quarteroni et al., 2015).

Definition 2.10 (Proper Orthogonal Decomposition). Assuming $N_{\text{data}} < \infty$, we define by $U_j = [u(\mu_j, t_k)]_{k=1}^{N_t} \in \mathbb{R}^{N_h \times N_t}$ the snapshot matrix for a given $\mu_j \in \mathcal{P}$ and consequently $U = [U_j]_{j=1}^{N_s} \in \mathbb{R}^{N_h \times N_{\text{data}}}$ the collective snapshot matrix. Then the (discrete) correlation matrix is given by

$$K = \frac{|\mathcal{P} \times \mathcal{T}|}{N_{\text{data}}} U U^T \in \mathbb{R}^{N_h \times N_h}. \quad (2.1)$$

K is symmetric and semi-positive definite and thus has real and positive eigenvalues $\sigma_1^2 \geq \dots \geq \sigma_r^2$. We further call ψ_k the eigenvectors of the correlation matrix K . With these, the corresponding POD basis of dimension $N < N_h$ is given via

$$\xi_k = \frac{1}{\sigma_k} U \psi_k, \quad 1 \leq k \leq N,$$

for the N largest eigenvalues and their corresponding eigenvectors. Then stacking these eigenvectors gives us the *POD matrix*

$$\mathbb{A} := [\xi_k]_{k=1}^N \in \mathbb{R}^{N_h \times N}.$$

Further, if $N_s, N_t = \infty$, we define the *continuous correlation matrix* via

$$K_\infty = \int_{\mathcal{P} \times \mathcal{T}} u(\mu, t) u^T(\mu, t) d(\mu, t) \in \mathbb{R}^{N_h \times N_h}, \quad (2.2)$$

and its real eigenvalues by $\sigma_{k,\infty}^2$.

Algorithm 1 Proper Orthogonal Decomposition (POD)

- 1: **Input:** snapshot matrix $U = [U_j]_{j=1}^{N_s} \in \mathbb{R}^{N_h \times N_{\text{data}}}$, reduction $N < N_h$
 - 2: **Output:** POD basis matrix $\mathbb{A} = [\xi_1, \dots, \xi_N] \in \mathbb{R}^{N_h \times N}$
 - 3: **if** $N_{\text{data}} \leq N_h$ **then**
 - 4: Form the correlation matrix $C = \frac{[\mathcal{P} \times \mathcal{T}]}{N_{\text{data}}} U^T U \in \mathbb{R}^{N_{\text{data}} \times N_{\text{data}}}$
 - 5: Solve the eigenvalue problem $C\psi_i = \sigma_i^2 \psi_i$, $i = 1, \dots, r$
 - 6: Set $\xi_i = \frac{1}{\sigma_i} U \psi_i$, $i = 1, \dots, r$
 - 7: **else**
 - 8: Form the correlation matrix $K = \frac{[\mathcal{P} \times \mathcal{T}]}{N_{\text{data}}} U U^T \in \mathbb{R}^{N_h \times N_h}$
 - 9: Solve the eigenvalue problem $K\xi_i = \sigma_i^2 \xi_i$, $i = 1, \dots, r$
 - 10: **end if**
 - 11: $\mathbb{A} \leftarrow \text{stack}(\xi_1, \dots, \xi_N)$
-

include theorem 6.1 from quarteroni2015rb

Proposition 2.11. Let K and K_∞ and their respective eigenvalues be defined as in Definition (2.10), then letting $N_s, N_t \rightarrow \infty$, we get

$$\sum_{k>N} \sigma_k^2 \longrightarrow \sum_{k>N} \sigma_{k,\infty}^2, \quad \text{a.s.}$$

Proof. Let $N_s, N_t < \infty$, conforming to the sampling assumption, and

$$M := \text{ess sup}_{(\mu, t) \in \Theta \times \mathcal{T}} \|u(\mu, t)\| < \infty$$

as before (see introduction in this Section (2.2.3)). Taking a variational approach on this finite dimensional space, we know, that there exists a matrix $V_\infty \in \mathbb{R}^{N_h \times N}$ such that

$$\begin{aligned} \sum_{k>N} \sigma_{k,\infty}^2 &= \int_{\mathcal{P} \times \mathcal{T}} \|u(\mu, t) - V_\infty V_\infty^T u(\mu, t)\|^2 d(\mu, t) \\ &= \min_{W \in \mathbb{R}^{N_h \times N}: W^T W = I} \int_{\mathcal{P} \times \mathcal{T}} \|u(\mu, t) - W W^T u(\mu, t)\|^2 d(\mu, t). \end{aligned}$$

Thus, since this is the optimal choice of an approximation matrix for the continuous case and $\mathbb{A} \in \mathbb{R}^{N_h \times N}$ given by Definition (2.10) is only optimal in the discrete setting and not necessarily the continuous case, we have

$$\sum_{k>N} \sigma_{k,\infty}^2 \leq \int_{\mathcal{P} \times \mathcal{T}} \|u(\mu, t) - \mathbb{A} \mathbb{A}^T u(\mu, t)\|^2 d(\mu, t) = \sum_{k>N} \sigma_k^2.$$

We then write entry-wise for $m, l = 1, \dots, N_h$

$$[K_\infty - K]_{ml} := \int_{\mathcal{P} \times \mathcal{T}} [u(\mu, t)]_m [u(\mu, t)]_l d(\mu, t) - \frac{|\mathcal{P} \times \mathcal{T}|}{N_{data}} \sum_{j=1}^{N_s} \sum_{k=1}^{N_t} [u(\mu_j, t_k)]_m [u(\mu_j, t_k)]_l.$$

By the uniform iid sampling assumed and the general bound on $\|u(\mu, t)\| \leq M$ before, we know that with the entry-wise boundedness of the PDE solutions, the resulting matrix is subject to the strong law of large numbers, and thus converges to zero, given the entry-wise matrix norm $\|\cdot\|_{\text{op}}$ as $N_t, N_s \rightarrow \infty$ almost surely.

Now using Bauer-Fike's Theorem (2.1) with the $\|\cdot\|_{\text{op}}$ -norm, we have upon ordering, that for each $\sigma_{k,\infty}^2$ there is a σ_k^2 in the spectrum of K , such that

$$|\sigma_k^2 - \sigma_{k,\infty}^2| \leq \kappa(\mathbb{A}_\infty) \|K - K_\infty\|_{\text{op}},$$

where $\kappa(\mathbb{A}_\infty)$ denotes the condition number of the stacked eigenvector matrix of K_∞ , i.e. $\mathbb{A}_\infty = (\xi_{1,\infty}, \dots, \xi_{N_h,\infty})$. This number is independent of N_s and N_t . Now by convergence of $\|K - K_\infty\|_{\text{op}} \rightarrow 0$ almost surely for $N_s, N_t \rightarrow \infty$, we conclude the proof. $\square$

Proposition 2.12. Let $\mathcal{V}_N = \{W \in \mathbb{R}^{N_h \times N} \mid W^T W = I_N\}$ be the set of all N -dimensional orthonormal bases, $N_{data} := N_t N_s$ and $\mathbb{A} = [\xi]_{i=1}^N$ be the POD matrix according to Definition (2.10), then

$$\begin{aligned} \frac{|\mathcal{P} \times \mathcal{T}|}{N_{data}} \sum_{i \in I, k=1, \dots, N_t} \|u(\mu_j, t_k) - \mathbb{A} \mathbb{A}^T u(\mu_j, t_k)\|_2^2 \\ = \min_{W \in \mathcal{V}_N} \frac{|\mathcal{P} \times \mathcal{T}|}{N_{data}} \sum_{i \in I, k=1, \dots, N_t} \|u(\mu_j, t_k) - W W^T u(\mu_j, t_k)\|_2^2 = \sum_{j=N+1}^r \sigma_k^2. \end{aligned}$$

Definition 2.13 (Linear Kolmogorov N -width). Let $N \in \mathbb{N}$, $N \leq N_h$ and $\mathcal{S}_{N_h} = \{u(\mu, t) \in \mathbb{R}^{N_h} \mid (\mu, t) \in \mathcal{P} \times \mathcal{T}\}$ be some manifold. The linear Kolmogorov N -width of $\mathcal{S}_{N_h}$ is defined as

$$d_N(\mathcal{S}_{N_h}) = \inf_{V \in \mathbb{R}^{N_h \times N}} \sup_{u \in \mathcal{S}_{N_h}} \|u - V V^T u\|.$$

Definition 2.14 (Nonlinear Kolmogorov N -width). Let $N \in \mathbb{N}$, $N \leq N_h$ and $\mathcal{S}_{N_h} = \{u(\mu, t) \in \mathbb{R}^{N_h} \mid (\mu, t) \in \mathcal{P} \times \mathcal{T}\}$ be some manifold. The nonlinear Kolmogorov N -width of $\mathcal{S}_{N_h}$ is defined as

$$\delta_N(\mathcal{S}_{N_h}) = \inf_{\psi \in C(\mathbb{R}^{N_h}, \mathbb{R}^N), \psi' \in C(\mathbb{R}^N, \mathbb{R}^{N_h})} \sup_{u \in \mathcal{S}_{N_h}} \|u - \psi(\psi'(u))\|.$$

2.3 Neural Networks

In this section, we will introduce the general notion of neural networks. Specifically important for us will be concatenations of perceptrons, which are a specific subclass of networks, the so called *feedforward neural network*.

Definition 2.15 (Neural Network). Let $d, L \in \mathbb{N}$, then a *neural network* Φ with input dimension d and L layers is defined via

$$\Phi = ((A_1, b_1), (A_2, b_2), \dots, (A_L, b_L)),$$

such that $N_0 = d$ and $N_1, \dots, N_L \in \mathbb{N}$, where each A_l is an $N_l \times \sum_{k=0}^{l-1} N_k$ dimensional matrix, and $b_l \in \mathbb{R}^{N_l}$.

We further define $R_\rho(\Phi) : \mathbb{R}^d \rightarrow \mathbb{R}^{N_L}$ as a *realizations of Φ with activation function ρ* for some arbitrary map $\rho : \mathbb{R} \rightarrow \mathbb{R}$ as the result of the following scheme for some input $x \in \mathbb{R}^d$:

$$\begin{aligned} x_0 &:= x, \\ x_l &:= \rho \left(A_l \begin{bmatrix} x_0^T & \cdots & x_{l-1}^T \end{bmatrix}^T + b_l \right), \quad \text{for } l = 1, \dots, L-1, \\ x_L &:= A_L \begin{bmatrix} x_0^T & \cdots & x_{L-1}^T \end{bmatrix}^T + b_L, \end{aligned}$$

where ρ acts componentwise, i.e. $\rho(y) := [\rho(y_1), \dots, \rho(y_m)]^T \in \mathbb{R}^m$ for each $y = [y_1, \dots, y_m] \in \mathbb{R}^m$. Note that we can write

$$A_l = \begin{bmatrix} A_{l,x_0} & \cdots & A_{l,x_{l-1}} \end{bmatrix},$$

with A_{l,x_k} being a $N_l \times N_k$ dimensional matrix for $k = 0, \dots, l-1$ and $l = 1, \dots, L$. This comes in handy to portray the scheme via

$$\begin{aligned} x_l &:= \rho \left(A_{l,x_0}x_0 + \cdots + A_{l,x_{l-1}}x_{l-1} + b_l \right), \quad \text{for } l = 1, \dots, L-1, \\ x_L &:= A_{L,x_{L-1}}x_{L-1} + b_L. \end{aligned}$$

Moreover call $N_\Phi := d + \sum_{l=1}^L N_l$ the *number of neurons* of the network, $L_\Phi := L$ the *number of layers*, and finally $\omega_\Phi := \sum_{l=1}^L (\|A_l\|_{\ell^0} + \|b_l\|_{\ell^0})$ the *number of active weights*.

Remark 2.16. As a note, we can rewrite known neural network architectures, such as a convolutional autoencoder, as a neural network according to Definition (2.15).

Motivate this with some example.

From this we can infer a simpler architecture, describing a network which omits any connection of non-neighboring layers.

Definition 2.17 (Standard Neural Network). If a neural network $\Phi = ((A_1, b_1), \dots, (A_L, b_L))$ now has the characteristic

$$A_{l,x_i} = 0, \quad \text{for } l = 1, \dots, L, \text{ and } i = 0, \dots, l-2,$$

then we call Φ a *standard neural network*.

When considering the similarities and differences between two distinct neural networks $\Phi_1 = ((A_1, b_1), \dots, (A_L, b_L))$ and $\Phi_2 = ((\hat{A}_1, \hat{b}_1), \dots, (\hat{A}_L, \hat{b}_L))$, one can easily see, that if $[A_1]_{ij} = 0$, while $[\hat{A}_1]_{ij} \neq 0$ for some i, j , then these networks are severely different, whereas if they only differ in the specific non-zero weight, they could be recovered from each other through training. This inspires the definition of *an architecture of a network*.

Definition 2.18 (Neural Network Architecture). For some $d, L \in \mathbb{N}$, we define a *neural network architecture $\mathcal{A}$ with input dimension d and L layers* to be a binary neural network, i.e. for all entries of $A_i, b_i \in \{0, 1\}$ for $i = 1, \dots, L$ of $\mathcal{A}$.

We further say that a *neural network $\Phi = ((\hat{A}_1, \hat{b}_1), \dots, (\hat{A}_L, \hat{b}_L))$ has architecture $\mathcal{A}$* , if

1. $N_l(\Phi) = N_l$ for all $l = 1, \dots, L$, and
2. $[\hat{A}_l]_{ij} \neq 0$ implies $[A_l]_{ij} \neq 0$ for all $l = 1, \dots, L$ with $i = 1, \dots, N_l$ and $j = 1, \dots, \sum_{k=0}^{l-1} N_k$.

Since the associated realization of a neural network $R_\rho(\Phi)$ requires the definition of ρ , the activation function, we will define the most commonly used one in literature. It will help a lot with some of the constructions, since it has a simple nature, but also provides the network with the necessary non-linearity.

Definition 2.19 (ReLU). The function

$$\rho : \mathbb{R} \rightarrow \mathbb{R}; \quad x \mapsto \max(0, x),$$

is called *ReLU* (Rectified Linear Unit) *activation function*.

Given such an activation function, Kurt Hornik provided in 1990 the first contribution to a field, which would emerge centuries later: Approximation theory. He proved for any $L^p(\mu)$ function, for μ some finite measure, the existence of an arbitrarily close ReLU valued neural network (Hornik, 1991).

Theorem 2.20 (Hornik's Theorem). *Define for some continuous, unbounded and non-constant actiation function ρ the set of all feedforward neural networks, as*

$$\mathcal{N}_d(\rho) := \bigcup_{L=1}^{\infty} \left\{ R_\rho(\Phi) : \mathbb{R}^d \rightarrow \mathbb{R} \mid \Phi \text{ is neural network with } L \text{ layers} \right\}.$$

Then $\mathcal{N}_d(\rho)$ lies dense in $L^p(\mu)$ for every finite measure μ , $p \in (0, \infty)$.

The proof of this statement leverages on Hahn Banach, but thus does not provide the specific architecture of the required neural network, which would be used for such an approximation. We aim significantly higher, namely at a bound on the necessary non-zero weights and layers for such an approximation, and will thus not mention the proof beyond this outline.

2.3.1 Network concatenations

In this subsection we will discuss the concatenations of multiple neural networks and the effects on the characteristic numbers of it. The complexity of a concatenated network can be given nicely, when the used activation function is

Definition 2.21 (Network Concatenation). Let $L_1, L_2 \in \mathbb{N}$ and

$$\Phi^1 = ((A_1, b_1), \dots, (A_{L_1}, b_{L_1})), \quad \Phi^2 = ((B_1, c_1), \dots, (B_{L_2}, c_{L_2})),$$

be two neural networks such that the input layer of Φ^1 has the same dimension as the output layer of Φ^2 . Then, $\Phi^1 \bullet \Phi^2$ denotes the following $L_1 + L_2 - 1$ layered network:

$$\Phi^1 \bullet \Phi^2 := \left(((B_1, c_1), \dots, (B_{L_2-1}, c_{L_2-1})), ((\tilde{A}_1, \tilde{b}_1), \dots, (\tilde{A}_{L_1}, \tilde{b}_{L_1})) \right),$$

where

$$\tilde{A}_l^1 := [A_{l,x_0} \ B_{L_2} \mid A_{l,x_1} \mid \dots \mid A_{l,x_{l-1}}], \quad \text{and} \quad \tilde{b}_l^1 := A_{l,x_0} c_{L_2} + b_l,$$

for $l = 1, \dots, L_1$. We call $\Phi^1 \bullet \Phi^2$ the *concatenation* of Φ^1 and Φ^2 .

Lemma 2.22 (Neutral Neural Network). Let $\Phi^{\text{Id}} = ((A_1, b_1), (A_2, b_2))$ be a d -dimensional neural network, with

$$A_1 := \begin{bmatrix} \text{Id}_{\mathbb{R}^d} \\ -\text{Id}_{\mathbb{R}^d} \end{bmatrix}, \quad b_1 := 0, \quad A_2 := \begin{bmatrix} 0_{\mathbb{R}^{d \times d}} & \text{Id}_{\mathbb{R}^d} \\ \text{Id}_{\mathbb{R}^d} & -\text{Id}_{\mathbb{R}^d} \end{bmatrix}, \quad b_2 := 0.$$

Then $R_\rho(\Phi^{\text{Id}}) = \text{Id}_{\mathbb{R}^d}$.

Lemma 2.23. For any two neural networks Φ^1 and Φ^2 according to Definition (2.21), $\Phi^1 \odot \Phi^2 := \Phi^1 \bullet \Phi^{\text{Id}} \bullet \Phi^2$ is a neural network with d -dimensional input, $L_1 + L_2$ layers with $\omega_{\Phi^1 \odot \Phi^2} \leq 2\omega_{\Phi^1} + 2\omega_{\Phi^2}$ active weights, $N_{\Phi^1 \odot \Phi^2} \leq 2N_{\Phi^1} + 2N_{\Phi^2}$ neurons. Additionally for all $x \in \mathbb{R}^d$, we have

$$R_\rho(\Phi^1 \odot \Phi^2)(x) = R_\rho(\Phi^1) \circ R_\rho(\Phi^2)(x).$$

Proof.

maybe include proof?

Intuitively speaking the latter network must account for all "missing" links, which a direct concatenation would necessarily be missing; any layer after the "switch" from the first to the latter needs to have a correction term in the weights, additionally attributing all potential connections which the first network would impose.

Let w.l.o.g. $L_2 = 2$, since the pass through the first layers of the combined network, is equivalent to the pass through the first layers of the individual network. Then we can show by induction:

1. Base case: $L_2 = 1$. We argue with $A_{L_1}x = A_{L_1, x_0}x$ for x of output dimension of Φ^1 , since the input dimension of Φ^1 is the output of Φ^2 .

$$\begin{aligned} R_\rho(\Phi^1) \circ R_\rho(\Phi^2)(x_0) &= A_{L_1} (B_{L_2, x_0}x_0 + B_{L_2, x_1}x_1 + c_{L_2}) + b_{L_1} \\ &= A_{L_1, x_0}B_{L_2, x_0}x_0 + A_{L_1, x_0}B_{L_2, x_1}x_1 + (A_{L_1, x_0}c_{L_2} + b_{L_1}) \\ &= A_{L_1, x_0}B_{L_2} \begin{bmatrix} x_0^T \\ x_1^T \end{bmatrix}^T + (A_{L_1, x_0}c_{L_2} + b_{L_1}) \\ &= R_\rho(\Phi^1 \bullet \Phi^2)(x_0) \end{aligned}$$

2. Induction assumption: Let the statement be true for $L_2 \geq 2$ fixed.
3. Induction step: $L_2 \mapsto L_2 + 1$.

□

2.3.2 Network parallelization

We have shown, that there is a possibility to simply compose networks, as is possible with functions. To further build upon that idea, we will now introduce stacking in the context of defining a network $P(\Phi^1, \Phi^2)$ from two networks Φ^1 and Φ^2 such that $R_\rho(P(\Phi^1, \Phi^2))(x) = (R_\rho(\Phi^1)(x), R_\rho(\Phi^2)(x))$ for all $x \in \mathbb{R}^d$.

Definition 2.24 (Network Parallelization). Let $L_1, L_2 \in \mathbb{N}$ s.t. $L_1 \leq L_2$ and let

$$\Phi^1 = ((A_1, b_1), \dots, (A_{L_1}, b_{L_1})), \quad \Phi^2 = ((B_1, c_1), \dots, (B_{L_2}, c_{L_2})),$$

be two neural networks with d -dimensional input. We call

$$P(\Phi^1, \Phi^2) := ((C_1, d_1), \dots, (C_{L_2}, d_{L_2})),$$

the *parallelization of Φ^1 and Φ^2* with

$$C_l := \left[\begin{array}{c|cc|ccc} A_{l,x_0} & A_{l,x_1} & 0 & \cdots & A_{l,x_{l-1}} & 0 \\ B_{l,x_0} & 0 & B_{l,x_1} & \cdots & 0 & B_{l,x_{l-1}} \end{array} \right], \quad d_l := \begin{bmatrix} b_l \\ c_l \end{bmatrix}, \quad \text{for } 1 \leq l < L_1,$$

$$C_{L_1} := \left[\begin{array}{c|cc|ccc} B_{L_1,x_0} & 0 & B_{L_1,x_1} & \cdots & 0 & B_{L_1,x_{L_1-1}} \\ B_{L_1,x_0} & 0 & B_{L_1,x_1} & \cdots & 0 & B_{L_1,x_{L_1-1}} \end{array} \right], \quad d_{L_1} := c_{L_1},$$

for $L_1 \leq l < L_2$, and

$$C_{L_2} := \left[\begin{array}{c|cc|ccc|cc|ccc} A_{L_2,x_0} & A_{L_2,x_1} & 0 & \cdots & A_{L_2,x_{L_2-1}} & 0 & 0 & \cdots & 0 \\ B_{L_2,x_0} & 0 & B_{L_2,x_1} & \cdots & 0 & B_{L_2,x_{L_2-1}} & B_{L_2,x_{L_2}} & \cdots & B_{L_2,x_{L_2-1}} \end{array} \right],$$

$$d_{L_2} := \begin{bmatrix} b_{L_2} \\ c_{L_2} \end{bmatrix}.$$

A similar construction is used for the case $L_1 > L_2$.

Lemma 2.25. In the setting of Definition (2.24), we have that $P(\Phi^1, \Phi^2)$ is a neural network with d -dimensional input, $\max\{L_1, L_2\}$ layers, $\omega_{P(\Phi^1, \Phi^2)} = \omega_{\Phi^1} + \omega_{\Phi^2}$ active weights and $N_{P(\Phi^1, \Phi^2)} = N_{\Phi^1} + N_{\Phi^2} - d$ neurons. Additionally, for all $x \in \mathbb{R}^d$, we have that

$$R_\rho(P(\Phi^1, \Phi^2))(x) = (R_\rho(\Phi^1)(x), R_\rho(\Phi^2)(x)).$$

maybe include proof?

2.3.3 Standard Neural Networks

Corollary 2.26. Let ρ be the ReLU activation function. Then there are constants $C_1, C_2 > 0$ such that for any neural network Φ with d -dimensional input and $L = L(\Phi)$ layers, $N = N(\Phi)$ neurons and $M = M(\Phi)$ nonzero weights, there is a standard neural network Φ_{st} with d -dimensional input, L layers, at most $C_1 LN$ neurons and $C_2 \cdot (LN + M)$ nonzero weights such that

$$R_\rho(\Phi)(x) = R_\rho(\Phi_{\text{st}})(x)$$

for all $x \in \mathbb{R}^d$.

2.3.4 Approximation Theory

For all further theoretical results we will assume that ρ , the activation function, is the ReLU as given in Definition (2.19) and furthermore define the set of functions bounded in Sobolev norm as

$$\mathcal{F}_{n,d,p,B} := \{f \in W^{n,p}((0,1)^d) \mid \|f\|_{W^{n,p}((0,1)^d)} \leq B\}.$$

To create a decent outline for this subsection, we will shortly discuss the goal and the steps to be taken; in order to show a bound on the weights and layers needed for an approximating neural network, we need to take into account the interpolation error, we may overcome with sufficient regularity. Say we would have a function f in the Sobolev space $W^{2,2}((0,1)^d)$, then it is apparent, that at least 2 derivatives of f must exist, in a piecewise, embedded sense, which can be leveraged to get a Taylor expansion around selected points.

This procedure will deduce a quantifiable error for the network approximating the initial arbitrary Sobolev function f in Sobolev norm, as long as we can bound the error, which the polynomial approximation produces with regard to the network itself.

This leaves us with two main objectives:

1. Bound the Sobolev function at critical equidistant points, using its Taylor expansion.
2. Relate a fixed error rate of the approximation polynomial and some neural network, to the networks complexity.

These two objectives are pursued by both Yarotsky (Yarotsky, 2017) and Gühring et al (Gühring et al., 2019), even though it is worth noting, that Yarotsky only provides the special case $s = 0, B = 1$, since the proof only shows an estimation with a Taylor expansion regarding the L^∞ norm, missing the crucial step of the interpolation bound to change the norm to the Sobolev expanded counterpart and additionally uses the L^∞ norm to approximate multiplication by a neural network.

Tho we will use the statement of Gühring later on, which is the extension of Yarotsky to any choice of s, p and B , we will only give the idea of the proof, since it follows the same pattern as Yarotsky's proof, but does include a lot of further theoretical setting, which will not be of much use in the actual results presented in this paper.

Proposition 2.27. There are constants $c_1, c_2, c_3 > 0$, such that for all $\varepsilon \in (0, 1)$ there is a neural network $\Phi_\varepsilon^{\text{sq}}$ with at most $c_1 \cdot \ln(1/\varepsilon)$ nonzero weights, at most $c_2 \cdot \ln(1/\varepsilon)$ layers, at most $c_3 \cdot \ln(1/\varepsilon)$ neurons, and with a 1-dimensional input and output such that for $f(x) := x^2$

$$\|R_\rho(\Phi_\varepsilon^{\text{sq}}) - f\|_{W^{0,\infty}((0,1)^1)} \leq \varepsilon,$$

and $R_\rho(\Phi_\varepsilon^{\text{sq}}(0)) = 0$.

Proof. Define the "tooth" function $g : [0, 1] \rightarrow [0, 1]$ with

$$g(x) = \begin{cases} 2x, & \text{for } x < 1/2, \\ 2(1-x), & \text{for } x \geq 1/2, \end{cases}$$

and its composition

$$g_s(x) := \underbrace{g \circ g \circ \dots \circ g}_{s\text{-times}}(x).$$

Furthermore, from the definition of the ReLU function (refer to (2.19)), we can also set $g(x) = 2\rho(x) - 4\rho(x - 1/2) + 2\rho(x - 1)$, which is intern, the same as defining a neural network with 1-dimensional input and output via $\Phi = ((A_1, b_1), (A_2, b_2))$ with

$$A_1 = \begin{bmatrix} \text{Id}_{\mathbb{R}} \\ \text{Id}_{\mathbb{R}} \\ \text{Id}_{\mathbb{R}} \end{bmatrix}, \quad b_1 = \begin{pmatrix} 0 \\ -1/2 \\ -1 \end{pmatrix}, \quad A_2 = \begin{bmatrix} 2 & -4 & 2 \end{bmatrix}, \quad b_2 = 0.$$

Then it is evident, using Lemma (2.23), that $g_s(x) = R_\rho(\underbrace{\Phi \odot \dots \odot \Phi}_{s\text{-times}})(x)$ for all $x \in \mathbb{R}$ represents a network with $\mathcal{O}(s)$ layers, active weights and neurons. Now, with an explicit formulation of g_m for a fixed level $m \geq 1$, one can identify the function $f(x) := x^2$ using linear interpolation on equidistant points $k/(2^m)$, where $k = 0, \dots, 2^m - 1$, i.e. let $\Phi_m = \Phi \odot \dots \odot \Phi$ (m -times) be a neural network, because then for all $x \in [\frac{k}{2^m}, \frac{k+1}{2^m}]$ it is true, that

$$R_\rho(\Phi_m)(x) = \left(\frac{(k+1)^2}{2^m} - \frac{k^2}{2^m} \right) \left(x - \frac{k}{2^m} \right) + \left(\frac{k}{2^m} \right)^2.$$

Why?

Then we also know that, using the Bramble-Hilbert lemma in one-dimension (Bramble & Hilbert, 1970), the fact that $f''(x) = 2$ for all $x \in (0, 1)$, and $h = 1/2^m$, that

$$\|R_\rho(\Phi_m) - f\|_{L^\infty((0,1))} \leq \max_{k=0,\dots,2^m-1} Ch^2 \|f''\|_{L^\infty([\frac{k}{2^m}, \frac{k+1}{2^m}])} = C' 2^{-2m}.$$

Hence setting $m = \lceil \ln(1/\varepsilon) \rceil$, with $C, C' > 0$ constants, only dependent on the domain, we achieve the wanted bound from the statement for the neural network $\Phi_\varepsilon^{\text{sq}} := \Phi_m$ with $\mathcal{O}(m) = \mathcal{O}(\ln(1/\varepsilon) + 1) \leq \mathcal{O}(2\ln(1/\varepsilon))$ layers, neurons and active weights. $\square$

This statement enables us to express multiplication using a neural network. This is straight forward, when viewing the polarization identity:

$$xy = \frac{1}{2}((x+y)^2 - x^2 - y^2), \quad \text{for } x, y \in \mathbb{R}, \quad (2.1)$$

as given by the first binomial formula.

Proposition 2.28. Let $M > 0$ and $\varepsilon \in (0, 1)$, then there is a ReLU neural network $\tilde{\times}$ with 2-dimensional input and 1-dimensional output, and constants $c_1 > 0$ and $c_2 = c_2(M) > 0$, such that for $\times : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}; (x, y) \mapsto xy$

1. $\|R_\rho(\tilde{\times}) - \times\|_{W^{0,\infty}((-M,M)^2)} \leq \varepsilon$,
2. if $x = 0$ or $y = 0$, then $R_\rho(\tilde{\times})(x, y) = 0$,
3. the number of layers, active weights and neurons of $\tilde{\times}$ is at most $c_1 \ln(1/\varepsilon) + c_2$.

Proof. Let $\delta := \varepsilon/(6M^2)$. Then we construct an approximation of the square function $f(x) = x^2$ for $x \in (0, 1)$ using Proposition (2.27) for the chosen δ , i.e. we define Φ_δ^{sq} , such that

$$\|R_\rho(\Phi_\delta^{\text{sq}}) - f\|_{W^{0,\infty}((0,1)^1)} \leq \frac{\varepsilon}{6M^2},$$

which now of course has $\mathcal{O}(\ln(1/\delta))$ many layers, active weights and neurons. Since the statement necessarily needs to implement negative numbers as well, we show with $|x| = \rho(x) + \rho(-x)$ that it is indeed possible to expand any neural network with a positive domain to a negative domain using a neural network (by simply concatenating). Thus we construct $\tilde{\times}$ via adding an addition layers on top of parallel square networks with scaled input, achieving

$$R_\rho(\tilde{\times})(x, y) = 2M^2 \left(R_\rho(\Phi_\delta^{\text{sq}}) \left(\frac{|x+y|}{2M} \right) - R_\rho(\Phi_\delta^{\text{sq}}) \left(\frac{|x|}{2M} \right) - R_\rho(\Phi_\delta^{\text{sq}}) \left(\frac{|y|}{2M} \right) \right).$$

To appreciate the third statement of our Proposition, we may now put together the necessary complexity of this network using the constants in Proposition (2.27);

1. We have 2 layers for the implementation of the absolute value, 1 for the addition in the end, and $c_0 \ln(1/\delta)$ for the square function, thus $L_{\tilde{\times}} \leq c_0 \ln(1/\varepsilon) + c_0 \ln(6M^2) + 3$.
2. We need $4 + 2 + 2$ active weights for the first matrix to distribute each (x, y) onto the parallel networks together with 6 active weights for the superposing of the input into the $|\cdot|$ -function. Another 3 are required for the last layer adding all outputs. Of course each of the different computations for the artificial scaled square function must be in parallel, thus $\omega_{\tilde{\times}} \leq 3c_0 \ln(1/\varepsilon) + 3c_0 \ln(6M^2) + 17$.

3. A similar consideration gives us $9 = (2 + 1) \cdot 3$ neurons for the first two and the last layer, thus giving us $N_{\times} \leq 3c_0 \ln(1/\varepsilon) + 3c_0 \ln(6M^2) + 9$.

Finally taking the adequate maximum, and remarking, that c_0 does not depend on M , we get the wanted bound in (2).

To make the true multiplication match the form of our implemented network, using (2.1), we may rewrite for $x, y \in (-M, M)$

$$\begin{aligned} xy &= 4M^2 \left(\frac{x}{2M} \cdot \frac{y}{2M} \right) \\ &= 4M^2 \frac{1}{2} \left(\left(\frac{x}{2M} + \frac{y}{2M} \right)^2 - \left(\frac{x^2}{2M} \right)^2 - \left(\frac{y^2}{2M} \right)^2 \right) \\ &= 2M^2 \left(\left(\frac{|x+y|}{2M} \right)^2 - \left(\frac{|x|}{2M} \right)^2 - \left(\frac{|y|}{2M} \right)^2 \right). \end{aligned}$$

Now with the deviation from each of the square approximation we get,

$$\begin{aligned} \|R_\rho(\tilde{\times}) - \times\|_{L^\infty((0,1)^2)} &\leq 2M^2 \left(3 \cdot \sup_{\frac{|x|}{2M} : x \in (-M, M)} |R_\rho(\Phi_\delta^{\text{sq}})(\tilde{x}) - \tilde{x}^2| \right) \\ &\leq 6M^2\delta = \varepsilon. \end{aligned}$$

We note a few things happening here for clarity; it is possible to simply use the reverse triangle equality to get all three term errors added, and then put them together. This can be done, since the $L^\infty((0,1)^2)$ norm appreciates only the maximal value of the error across all possible elements of the $(0,1)^2$ domain, but because the inner function is bijective, these are just the same. Additionally we use the, trivial by definition, statement

$$\|g \circ f\|_{L^\infty((0,1))} \leq \|g\|_{L^\infty((0,1))}.$$

This has shown the statement (3).

Statement (1) on the other hand, is a direct consequence of the subliminally result in Proposition (2.27). $\square$

As a last step, before stating and proving the Yarotsky Theorem, we want to discuss the construction and complexity of a network approximating any collection of equidistantly spaced function with local support in a higher dimensional domain $(0,1)^d$ for $1 \leq d < \infty$. If we take into account, that we may also want these functions to form a partition of unity on the high dimensional domain, this will be ground to tackle the complexity of the approximation of polynomials with neural networks.

Lemma 2.29. For any $d, N \in \mathbb{N}$ there is a collection of functions

$$\Psi = \left\{ \phi_m \mid m \in \{0, \dots, N\}^d \right\},$$

with $\phi : \mathbb{R}^d \rightarrow \mathbb{R}$ for all $m = (m_1, \dots, m_d) \in \{0, \dots, N\}^d$ with the following properties:

1. $0 \leq \phi_m(x) \leq 1$ for all $\phi_m \in \Psi$ and every $x \in \mathbb{R}^d$,
2. $\sum_{\phi_m \in \Psi} \phi_m(x) = 1$ for every $x \in [0, 1]^d$,

3. $\text{supp}(\phi_m) \subset B_{1/N, \|\cdot\|_{\ell^\infty}}(m/N)$ for all $\phi_m \in \Psi$,
4. there are constants $C, c \geq 1$ such that for each $\phi_m \in \Psi$ there is a neural network Φ_m with d -dimensional input and output, at most 3 layers, $C \cdot d$ active weights and neurons, such that

$$\prod_{l=1}^d [R_\rho(\Phi_m)]_l = \phi_m$$

and $\| [R_\rho(\Phi_m)]_l \|_{L^\infty((0,1)^d)} \leq 1$ for all $l = 1, \dots, d$.

Proof. The idea of the proof, will be one of construction, where we will find adequate functions, which are easily transferable into ReLU neural networks. For this, define the functions

$$\psi : \mathbb{R} \rightarrow \mathbb{R}, \quad \psi(x) = \begin{cases} 1, & |x| < 1, \\ 0, & 2 < |x|, \\ 2 - |x| & 1 \leq |x| \leq 2, \end{cases}$$

and $\phi_m : \mathbb{R}^d \rightarrow \mathbb{R}$ as the product of shifted and scaled versions of ψ , which inherit its local support, i.e.

$$\phi_m(x) := \prod_{l=1}^d \psi \left(3N \left(x_l - \frac{m_l}{N} \right) \right),$$

for $m = (m_1, \dots, m_d) \in \{0, \dots, N\}^d$. This function satisfies our first three parts of the Lemma directly: Take one x_i for some $i = 1, \dots, d$. Now we get that $\psi(3N(x_i - (m_i/N))) \in [-2, 2]$, since ψ has support of $[-2, 2]$. This gives us

$$x \in \left[\frac{m}{N} - \frac{2}{3N}, \frac{m}{N} + \frac{2}{3N} \right],$$

yielding the third property, as we took i arbitrarily. Simultaneously we use the zero product property, and the fact, that the range of ψ is $[0, 1]$ to get the first property. Defining $z_i := 3Nx_i$, we can rewrite for any N the one-dimensional function

$$S(x_i) := \sum_{m_i=0}^N \psi \left(3N \left(x_i - \frac{m_i}{N} \right) \right) \quad \text{into the function} \quad \tilde{S}(z_i) := \sum_{m_i=0}^N \psi(z_i - 3m_i),$$

that intern gives us the wanted result of $\tilde{S}(z_i) = 1$ for any $z_i \in [0, 3N]$, because the least distance between points is 3, which means going forwards or backwards either lands one outside the support of the function, if $z_i \in [-1, 1]$, or it adds both third case values, i.e. setting $z_i \in [1, 2]$ w.l.o.g

$$\psi(z_i) + \psi(z_i - 3) = (2 - z_1) + (2 + (z_i - 3)) = 1.$$

Now we can extrapolate this result to S by a simple scaling argument and then use Fubini and the factorization of the sum over products to get

$$\sum_{m \in \{0, \dots, N\}^d} \phi_m(x) = \sum_{m_1=0}^N \cdots \sum_{m_d=0}^N \prod_{l=1}^d \psi \left(3N \left(x_l - \frac{m_l}{N} \right) \right) = \prod_{l=1}^d \left(\sum_{m_l=0}^N \psi \left(3N \left(x_l - \frac{m_l}{N} \right) \right) \right) = 1.$$

Finally we are only left with proving the forth property. For this, we need to construct a neural network Φ_ψ , that realizes ψ . Let

$$A_1 := \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad b_1 := \begin{pmatrix} 2 \\ 1 \\ -1 \\ -2 \end{pmatrix}, \quad A_2 := \begin{bmatrix} 0 & 1 & -1 & -1 & 1 \end{bmatrix}, \quad b_2 := 0,$$

define the neural network $\Phi_\psi = ((A_1, b_1), (A_2, b_2))$. With distinction by cases one can easily see, that for all $x \in \mathbb{R}$

$$\begin{aligned} R_\rho(\Phi_\psi)(x) &= A_2(\rho(A_1x + b_1)) + b_2 \\ &= A_2 \left(\begin{bmatrix} \rho(x+2) & \rho(x+1) & \rho(x-1) & \rho(x-2) \end{bmatrix}^T \right) \\ &= \rho(x+2) + \rho(x-2) - (\rho(x+1) + \rho(x-1)) \\ &= \psi(x). \end{aligned}$$

Furthermore the complexity of this network is 2 layers, 12 active weights and 6 neurons. Let $\Phi_{m,l}$ be a neural network with d -dimensional input and one-dimensional output, such that

$$R_\rho(\Phi_{m,l})(x) = 3N \left(x_l - \frac{m_l}{N} \right), \quad \text{for all } x \in \mathbb{R}^d \text{ and } m \in \{0, \dots, N\}^d.$$

Then the complexity of this network is 1 layer, 2 active weights and $d+1$ neurons. Finally realizing that the composition of both these functions, i.e. the realization of the concatenation of networks, would yield each factor for the wanted function. If we now parallelize this concatenation, we get the forth property by definition, i.e. let

$$\Phi_m := P(\Phi_\psi \odot \Phi_{m,l} \mid l = 1, \dots, d),$$

since then

$$\prod_{l=1}^d [R_\rho(\Phi_m)]_l(x) = \phi_m(x), \quad \text{for all } x \in \mathbb{R}^d.$$

This network now has d -dimensional input and output and the complexity, using both Lemma (2.23) and Lemma (2.25), of 3 layers, at most $(12+2) \cdot 2 \cdot d = 28d$ active weights, and at most $9d$ neurons. $\square$

With these auxiliary results, we may finally start with the statement and proof of the Yarotsky Theorem.

Theorem 2.30 (Yarotsky Theorem). *Let $d \in \mathbb{N}, n \in \mathbb{N}_{\geq 2}, B = 1$ and $f \in \mathcal{F}_{n,d,\infty,B}$ some arbitrary function. Then there is a constant $c = c(d, n) > 0$ with the following properties:*

For any $\varepsilon \in (0, 1)$, there is a neural network architecture $\mathcal{A}_\varepsilon = \mathcal{A}_\varepsilon(d, n, \varepsilon)$ with d -dimensional input and one-dimensional output such that, there is a neural network Φ_ε^f that has architecture $\mathcal{A}_\varepsilon$ and satisfies

$$\|f - R_\rho(\Phi_\varepsilon^f)\|_{W^{0,\infty}((0,1)^d)} \leq \varepsilon,$$

and

1. $L_{\mathcal{A}_\varepsilon} \leq c \cdot \ln(1/\varepsilon),$
2. $\omega_{\mathcal{A}_\varepsilon} \leq c \cdot \varepsilon^{-d/n} \ln(1/\varepsilon),$
3. $N_{\mathcal{A}_\varepsilon} \leq c \cdot \varepsilon^{-d/n} \ln(1/\varepsilon).$

Proof. Given the definition of the functions $\phi_m \in \Psi$ as constructed in the proof of Lemma (2.29), we want to structure the proof of this Theorem in two sub-goals:

The formulation of an approximate function f_1 for f using the partition of unity and an adequate Taylor expansion, and the subsequent approximation of that function f_1 with a neural network Φ_{f_1} using the $\tilde{\times}$ -multiplication of Proposition (2.28).

Step 1: Taylor Expansion

For any $m \in \{0, \dots, N\}^d$, consider the $(n-1)$ -Taylor polynomial for the function f at $x_0 = m/N$:

$$P_m(x) = \sum_{n: |n| < n} \frac{D^n f(x)}{n!} \Big|_{x=\frac{m}{N}} \left(x - \frac{m}{N}\right)^n, \quad \text{for } x \in \mathbb{R}^d,$$

with the conventions $n! = \prod_{l=1}^d n_l!$ and $(x - m/N)^n = \prod_{l=1}^d (x_l - \frac{m_l}{N})^{n_l}$. With the partition of unity of Lemma (2.29), we can then set any function, as a linear combination of its point-values and the partition function. Especially this allows us to define

$$f = \sum_{m \in \{0, \dots, N\}^d} \phi_m f, \quad \text{and} \quad f_1 = \sum_{m \in \{0, \dots, N\}^d} \phi_m P_m.$$

Now we can exploit the properties given by Lemma (2.29), as follows; firstly use the $\|\phi_m\|_{L_\infty((0,1)^d)} = 1$ property in Line (2.3), then one can estimate the maximal number of partitions with 2^d in Line (2.4). After this we take the simple Lagrange remainder

argue this

in Line (2.5). Lastly one can simply argue with $f \in \mathcal{F}_{n,d,\infty,B=1}$ in Line (2.6).

$$\|f - f_1\|_{L_\infty((0,1)^d)} \leq \sum_{m \in \{0, \dots, N\}^d} \|\phi_m (f - P_m)\|_{L_\infty((0,1)^d)} \quad (2.2)$$

$$\leq \sum_{m: |x_l - \frac{m_l}{N}| < \frac{1}{N}, \forall l=1, \dots, d} \|f - P_m\|_{L_\infty((0,1)^d)} \quad (2.3)$$

$$\leq 2^d \max_{m: |x_l - \frac{m_l}{N}| < \frac{1}{N}, \forall l=1, \dots, d} \|f - P_m\|_{L_\infty((0,1)^d)} \quad (2.4)$$

$$\leq \frac{2^d d^n}{n!} \left(\frac{1}{N}\right)^n \max_{n: |n|=n} \|D^n f\|_{L_\infty((0,1)^d)} \quad (2.5)$$

$$\leq \frac{2^d d^n}{n!} \left(\frac{1}{N}\right)^n \quad (2.6)$$

Now taking

$$N = \left\lceil \left(\frac{n!}{2^d d^n} \cdot \frac{\varepsilon}{2} \right)^{-\frac{1}{n}} \right\rceil$$

gives the approximation bound of

$$\|f - f_1\|_{L_\infty((0,1)^d)} \leq \frac{\varepsilon}{2}. \quad (2.7)$$

Step 2: Approximation by a Neural Network

The polynomial function f_1 may be further expressed in a different form, which allows us to separately compare it to a potential network approximation using the definition of ϕ_m , i.e. for $x \in \mathbb{R}^d$

$$f_1(x) = \sum_{m \in \{0, \dots, N\}^d} \sum_{n: |n| < n} a_{m,n} \underbrace{\left(\phi_m(x) \left(x - \frac{m}{N}\right)^n \right)}_{=: f_{m,n}(x)}. \quad (2.8)$$

This expansion now has at most $d^n(N+1)^d$ terms containing $f_{m,n}$, since

$$\left| \left\{ n \in \{0, \dots, n-1\}^d : |n| < n \right\} \right| \leq \left| \left\{ n \in \{0, \dots, n-1\}^d \right\} \right| = d^n,$$

and obviously $|\{m = \{0, \dots, N\}^d\}| = (N+1)^d$. We note, that we take $M = d+n$ for the definition of the network multiplication, i.e. $\tilde{\times} : (-M, M)^2 \rightarrow \mathbb{R}$. This is done, because we have as an obvious consequence from the fact, that if $x, y \in (-1, 1)$, that

$$|R_\rho(\tilde{\times})(x, y) - xy| \leq \delta \implies |R_\rho(\tilde{\times})(x, y)| \leq |y| + \delta,$$

for some fixed but not yet specified $\delta \in (0, 1)$. Recursively concatenating $\tilde{\times}$ for $(d+n-1)$ -times can then propagate the error to at most $M = (d+n)$ upon the last realization of $\tilde{\times}$.

As aforementioned, we can now compare each function $f_{m,n}$ pointwise to a realization of a network approximating at most $n-1$ of the multiplications needed for $\prod_{l=1}^d (x_l - (m_l/N))^{n_l}$, as we know that $\sum_{l=1}^d n_l < n$, d additional multiplications for the realization of the function ϕ_m and one multiplication for putting it together. Hence we define a neural network $\Phi_{f_1}^{m,n}$ with d -dimensional input and one dimensional output via

$$\begin{aligned} R_\rho(\Phi_{f_1}^{m,n})(x) = R_\rho(\tilde{\times}) & \left([R_\rho(\Phi_m)]_1(x_1), R_\rho(\tilde{\times}) \left([R_\rho(\Phi_m)]_2(x_2), \dots \right. \right. \\ & R_\rho(\tilde{\times}) \left([R_\rho(\Phi_m)]_d(x_d), R_\rho(\tilde{\times}) \left(x_1 - \frac{m_1}{N}, \dots \right. \right. \\ & R_\rho(\tilde{\times}) \left(x_{d-1} - \frac{m_{d-1}}{N}, R_\rho(\tilde{\times}) \left(x_d - \frac{m_d}{N}, \dots \right. \right. \\ & \left. \left. \left. \left. \left. \left. R_\rho(\tilde{\times}) \left(x_d - \frac{m_d}{N}, x_d - \frac{m_d}{N} \right) \right) \right) \right) \right) \right) \right). \end{aligned}$$

for $x = (x_1, \dots, x_d) \in \mathbb{R}^d$ where we used Φ_m as in the proof Lemma (2.29). This network contains at most $(d+n)$ concatenations of $\tilde{\times}$, and thus by Lemma (2.29) and its constant $C > 0$, Lemma (2.23), and Proposition (2.28) and its constants $c_0, c_1(d, n) > 0$,

$$L_{\Phi_{f_1}^{m,n}} \leq 3c_0(d+n) \left(\ln \left(\frac{1}{\delta} \right) + (d+n)c_1 \right), \quad \omega_{\Phi_{f_1}^{m,n}} \leq C2(d+n)d \left(\ln \left(\frac{1}{\delta} \right) + 2(d+n)C \right). \quad (2.9)$$

for a given precision level $\delta \in (0, 1)$, where the number of neurons are of the same order as the number of active weights.

The error comparison of $R_\rho(\Phi_{f_1}^{m,n})$ and $f_{m,n}$ can then be done by counting the maximal depth of usages of the network $\tilde{\times}$, which is $(d+n)$ as shown above, and the estimate of

$$\left| \left(x - \frac{m}{N} \right) \right| \leq 1, \quad |\psi(x)| \leq 1,$$

for all $x \in (0, 1)^d$. Then directly

$$\| R_\rho(\Phi_{f_1}^{m,n}) - f_{m,n} \|_{L_\infty((0,1)^d)} \leq (d+n)\delta.$$

Now since, as mentioned below Equation (2.8), f_1 has at most $d^n(N+1)^d$ terms, but since any $x \in \mathbb{R}^d$ can be only in the support of at most 2^d functions ϕ_m by the second statement in Proposition (2.28), the sum going over all m can be reduced to only those, which are non-zero, which we employ in Line (2.11). This brings $(N+1)^d \rightarrow 2^d$, supporting the equation in Line (2.12). Setting

$$\delta = \frac{\varepsilon}{2 \cdot 2^d d^n (d+n)}$$

allows us to finally assert, using the bound of Equation (2.9) for Line (2.13),

$$\|R_\rho(\Phi_{f_1}) - f_1\|_{L_\infty((0,1)^d)} \leq \sum_{m \in \{0, \dots, N\}^d} \sum_{n: |n| < n} |a_{m,n}| \|R_\rho(\Phi_{f_1}^{m,n}) - f_{m,n}\|_{L_\infty((0,1)^d)} \quad (2.10)$$

$$= \sum_{m: x \in \text{supp}(\phi_m)} \sum_{n: |n| < n} |a_{m,n}| \|R_\rho(\Phi_{f_1}^{m,n}) - f_{m,n}\|_{L_\infty((0,1)^d)} \quad (2.11)$$

$$\leq 2^d d^n \max_{m: x \in \text{supp}(\phi_m)} \max_{n: |n| < n} \|R_\rho(\Phi_{f_1}^{m,n}) - f_{m,n}\|_{L_\infty((0,1)^d)} \quad (2.12)$$

$$\leq 2^d d^n (d + n) \delta \quad (2.13)$$

$$= \frac{\varepsilon}{2}. \quad (2.14)$$

Our knowledge of δ together with at most $d^n(N+1)^d = \mathcal{O}(\varepsilon^{-d/n})$ parallel networks with one additive layer and the fact that generally we have $c' \ln(1/\delta + c'') \leq c''' \ln(1/\delta)$ for $c', c'' > 0$ and $c''' \geq c' \ln(1/c'')$, gets us the final complexity of Φ_{f_1} from the starting point of Equation (2.9), i.e.

$$L_{\Phi_{f_1}} \leq c \ln(1/\varepsilon), \quad \omega_{\Phi_{f_1}} \leq c \varepsilon^{-d/n} \ln(1/\varepsilon), \quad N_{\Phi_{f_1}} \leq c \varepsilon^{-d/n} \ln(1/\varepsilon).$$

with some constant $c = c(d, n) > 0$.

Using the triangle inequality, we can show that this network indeed satisfies the wanted precision bound with both errors stemming from Equation () and Equation () respectively:

$$\|f - R_\rho(\Phi_{f_1})\|_{L_\infty((0,1)^d)} \leq \|f - f_1\|_{L_\infty((0,1)^d)} + \|f_1 - R_\rho(\Phi_{f_1})\|_{L_\infty((0,1)^d)} \leq \frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon.$$

□

We may even expand Yarotsky to cover a more general case, where $f \in \mathcal{F}_{n,d,p,B}$ for arbitrarily $1 \leq p \leq \infty$ and $B > 0$ and the norm, to which we assert the precision, an arbitray Sobolev norm. This extension is presented by Gühring et al. (Gühring et al., 2019).

Theorem 2.31 (Gühring Theorem). *Let $d \in \mathbb{N}, n \in \mathbb{N}_{\geq 2}, B > 0$ and $0 \leq s \leq 1$ and $f \in \mathcal{F}_{n,d,p,B}$ some arbitrary function. Then there is a constant $c = c(d, n, p, B, s) > 0$ with the following properties:*

For any $\varepsilon \in (0, 1)$, there is a neural network architecture $\mathcal{A}_\varepsilon = \mathcal{A}_\varepsilon(d, n, p, B, s, \varepsilon)$ with d -dimensional input and one-dimensional output such that, there is a neural network Φ_ε^f that has architecture $\mathcal{A}_\varepsilon$ and satisfies

$$\|f - R_\rho(\Phi_\varepsilon^f)\|_{W^{s,p}((0,1)^d)} \leq \varepsilon,$$

and

1. $L_{\mathcal{A}_\varepsilon} \leq c \cdot \ln(1/\varepsilon),$
2. $\omega_{\mathcal{A}_\varepsilon} \leq c \cdot \varepsilon^{-d/(n-s)} \ln(1/\varepsilon),$
3. $N_{\mathcal{A}_\varepsilon} \leq c \cdot \varepsilon^{-d/(n-s)} \ln(1/\varepsilon).$

Proof Sketch. Considering that all of the tools used for this proof will neither be of any use to the main results in this paper nor to the general structure, we will only sketch the general steps necessary to achieve it for the interested reader:

1. The first step is to expand the Proposition (2.27) to the $W^{1,\infty}(0,1)$ -norm. This is done by simple calculation, since an explicit construction is being made.
2. This can be applied to Proposition (2.28) in the same way. Additionally we need to bound $|R_\rho(\tilde{x})|_{W^{1,\infty}((0,1)^2)}$, since this is required in for the bound on the collective error under the Taylor expanded function in the end.
3. The most import additional step is to bound the switch of norms from $W^{1,p}((0,1)^d)$ to $W^{s,p}((0,1)^d)$ of a function being approximated by the partition of unity, that we discussed in Lemma (2.29). This gives

$$\|f - f_N\|_{W^{s,p}((0,1)^d)} \leq C \left(\frac{1}{N}\right)^{n-s} \|f\|_{W^{n,p}((0,1)^d)},$$

for a partition of unity of size N . It is worth noting, that this factor in front is responsible for the different approximation in regard to weights and neurons, since it influences the necessary choice of δ .

4. The last step, which differs greatly from the proof of Yarotsky Theorem (2.30), is the approximation of a sum of localized polynomials with a neural network. Here the challenge lies again in the control of the derivatives.

□

Chapter 3

Formulation of the General Problem

Describe basic already existing techniques via literature

Introduce Proper Orthogonal Decomposition

Introduce Autoencoders

Introduce Deep Orthogonal Decomposition

3.1 Time-dependent parametric PDE

Definitely add traditional convergence and computational cost.

Let $\Theta \subset \mathbb{R}^p$ and $\Theta' \subset \mathbb{R}^q$ be compact and denote the geometric and physical parameter spaces respectively. Further we let $\Omega \subset \mathbb{R}^d$ be some Lipschitz domain and $[0, T]$ be a time interval for some $T > 0$ of a parametric time-dependent PDE, i.e. for any $\mu \in \Theta$ and $\nu \in \Theta'$

$$u^{\mu, \nu} := u(\mu, \nu), \quad \text{in a sufficiently smooth space,}$$

is the (classical) solution of the PDE, for the choice of the space depending on the given specified problem, if

$$\begin{cases} \frac{\partial}{\partial t} u^{\mu, \nu}(x, t) + \mathcal{L}(\mu, \nu) u^{\mu, \nu}(x, t) + \mathcal{N}(u^{\mu, \nu}(x, t), \mu, \nu) = f(x, t; \mu, \nu), & (x, t) \in \Omega \times [0, T], \\ \mathcal{B}(\mu, \nu) u^{\mu, \nu}(x, t) = g_N(x, t; \mu, \nu) & (x, t) \in \partial\Omega \times [0, T], \\ u^{\mu, \nu}(x, t) = u_0^{\mu, \nu}(x) & (x, t) \in \Omega \times \{0\}, \end{cases}$$

where $\mathcal{L}$ represents a linear, and $\mathcal{N}$ represents a non-linear integro-differential operator. Using calculus of variation or distribution theory, one can then formulate a weak form of the problem and find a general space, most commonly Sobolev (for elliptic problems with coercive properties), Bochner (for parabolic problems with coercive properties on the stationary part) or discontinuous Galerkin spaces (for hyperbolic problems with enough regularity), on which a weak solution can be defined. Such a space $\mathcal{H}$ is generally some Banach space, most often a subspace of L^2 .

This weak solution will have less regularity imposed than the strong formulation, but this sometimes allows us to retrieve existence and uniqueness results. Quite famously, there are counter examples of course; the Navier-Stokes Equation in three dimensions poses one such result, where a weak form exists and numerical methods have been developed, which can provide a solution in special cases, but existence remains unproven in general. As mentioned beforehand, we will

completely omit the exact transformation of the strong PDE formulation into a weak one, and simple assume that this can be done:

Say we (can) discretize the problem in the spacial domain Ω with N_h degrees of freedom, i.e. let $\mathcal{H}_h \subset \mathcal{H}$ be a Hilbert space with $\mathcal{H}_h \cong \mathbb{R}^{N_h}$, a scalar product $\langle \cdot, \cdot \rangle_{\mathcal{H}_h}$ and a corresponding norm $\| \cdot \| := \| \cdot \|_{\mathcal{H}_h}$ such that the resulting full order model solution can be identified using the formulation

$$u_h(x, t; \mu, \nu) = \sum_{j=1}^{N_h} [u_h^{\mu, \nu}(t)]_j \varphi_j(x) \in C^1([0, T]; \mathcal{H}_h) \quad (3.1)$$

with $\{\varphi_j\}_{j=1}^{N_h}$ the basis of $\mathcal{H}_h$ and $u_h^{\mu, \nu}(t) \in \mathbb{R}^{N_h}$ for each $t \in [0, T]$ and the underlying problem for the coefficients

$$\begin{cases} M(\mu, \nu) \frac{\partial}{\partial t} u_h^{\mu, \nu}(t) + A(\mu, \nu) u_h^{\mu, \nu}(t) + N(u_h^{\mu, \nu}(t), \mu, \nu) = f(t; \mu, \nu) & t \in [0, T], \\ u_h^{\mu, \nu}(0) = u_0(\mu, \nu) & t = 0, \end{cases} \quad (3.2)$$

where we assume

- $M : \Theta \times \Theta' \rightarrow \mathbb{R}^{N_h \times N_h}$ to send the parameter set to a mass matrix,
- $A : \Theta \times \Theta' \rightarrow \mathbb{R}^{N_h \times N_h}$ to send the set to a symmetric positive definite stiffness matrix,
- $N : \mathbb{R}^{N_h} \times \Theta \times \Theta' \rightarrow \mathbb{R}^{N_h}$ to be some non-linear term,
- $f : [0, T] \times \mathbb{R}^{N_h} \times \Theta \times \Theta' \rightarrow \mathbb{R}^{N_h}$ to be the corresponding source term of the equation.

Additionally let $u_0 : \Theta \times \Theta' \rightarrow \mathbb{R}^{N_h}$ be the initial data.

Note that it is indeed possible to arrive at that discretization without skipping to a weak formulation, e.g. using finite differences or finite volume schemes, but since any existence statement needs a more general space than C^n to guarantee completeness and hence imposes some identity using a basis, we need only to infer such a procedure. For the interested reader, an explicit discretization using finite volume schemes can be seen in (Haasdonk & Ohlberger, 2008).

This formulation establishes smoothness with regard to time, however we will implicitly further discretize in time using $\{t_k\}_{k=1}^{N_t} := \mathcal{T}$ time steps by gathering data only at these points. This is called *method of lines approach* (Hesthaven et al., 2022, p. 278).

Definition 3.1 (Solution Data). Following this, we may denote with

$$u_h^{\mu, \nu, t} := u_h^{\mu, \nu}(t) \in \mathbb{R}^{N_h}, \quad \text{for } \mu \in \Theta, \nu \in \Theta' \text{ and all } t \in \mathcal{T},$$

a finite dimensional *solution snapshot* and

$$u_h^{\mu, \nu} := [u_h^{\mu, \nu}(t)]_{t \in \mathcal{T}} \in \mathbb{R}^{N_h \times N_t}, \quad \text{for } \mu \in \Theta \text{ and } \nu \in \Theta',$$

with $N_t := |\mathcal{T}|$ a finite dimensional *solution trajectory* of the discretized parametric PDE. Furthermore, let $\mathcal{P}_1 \subset \Theta$, $\mathcal{P}_2 \subset \Theta'$ and $\mathcal{P} = \mathcal{P}_1 \times \mathcal{P}_2$ be some finite parameter data with respective sizes $N_{s_1}, N_{s_2} \in \mathbb{N} \cup \{\infty\}$ and $N_s = N_{s_1} N_{s_2}$.

Mainly, we stated the problem framework, to know with which class of PDEs we will be working with. Hence any regularity assumption is only given here in order to assert an outlook on the use of L^p norms for the error between the weak solution $u \in \mathcal{H}$, the finite dimensional solution $u_h \in \mathcal{H}_h$ and the approximation of that solution $\hat{u}_h \in \mathcal{H}_h$ (of reduced order) using ROM methods.

It is the latter ones, we will be concerned with in this thesis, which are however clearly embedded in L^p , since they are finite dimensional and bounded by an upcoming assumption.

From this point forward, we want to consistently rely on the following few assumptions, which regulate the discrete problem above. The first Assumption (1) justifies a nuanced approach regarding the geometric parameter μ and the physical parameter ν . The second Assumption (2) is necessary for the derivation of any convergence result to a true parameter-to-solution map, when sampling an ever growing number of examples, while the third Assumption (3) imposes regularity on the mentioned parameter-to-solution map. Interestingly enough, it is that Assumption, which ensures the existence of a true solution of the discrete problem for any specified parameter choice. Observing of course, that this does not translate into the solvability for the upstream general problem. The last Assumption (4) gives us the ability to later use an Autoencoder for a reduction.

Assumption 1. Let $\mathcal{S} := \{u_h^{\mu,\nu,t} \in \mathbb{R}^{N_h} \mid (\mu, \nu, t) \in \Theta \times \Theta' \times [0, T]\}$ denote the *total solution manifold* and $\mathcal{S}_{\mu,t} := \{u_h^{\mu,\nu,t} \in \mathbb{R}^{N_h} \mid \nu \in \Theta'\}$ denote the *solution submanifold* for a fixed set $(\mu, t) \in \Theta \times [0, T]$, then we assume that

1. the linear KnW of $\mathcal{S}$ decays slowly, i.e.

$$d_N(\mathcal{S}) \leq CN^{-\alpha},$$

where $0 < \alpha \leq 1$ and $C > 0$ some constant.

2. the linear KnW of $\mathcal{S}_{\mu,t}$ decays quickly for all $\mu \in \Theta$ and $t \in [0, T]$, i.e.

$$\sup_{(\mu,t) \in \Theta \times [0,T]} d_N(\mathcal{S}_{\mu,t}) \leq C'N^{-\beta},$$

where $\beta > 1$ and $C' > 0$ some constant.

This is mostly satisfied for problems in which the transport dominated or non-linear part is chained to the parameter μ , and any coercive or elliptic operator is most heavily influence by ν .

Assumption 2. Let Θ, Θ' and $[0, T]$ for $T > 0$ as above. We assume that all $N_s \in \mathbb{N}_{\geq 2}$ parameter data points according to Definition (3.1) are sampled uniformly and iid in the joint parameter space $\Theta \times \Theta'$, while a uniform grid is employed for the time variable, $\mathcal{T} = \{\Delta t, 2\Delta t, \dots, N_t \Delta t\}$, where $N_t \in \mathbb{N}_{\geq 2}$, N_{s_1} and N_{s_2} are the respective number of random samples in the parameter spaces Θ and Θ' , $N_s = N_{s_1} N_{s_2} \in \mathbb{N}$ is the total sample size, and $\Delta t = T/N_t$.

We remark, that the equidistant spacing of the time samples is a matter of convenience, and could be loosened under some constraints.

Assumption 3. Let $\mathcal{G} : \Theta \times \Theta' \times [0, T] \rightarrow \mathbb{R}^{N_h}$ be the parameter-to-solution map, mapping $(\mu, \nu, t) \mapsto u_h^{\mu,\nu,t}$. Here we assume that

1. $m = \operatorname{ess\,inf}_{(\mu,\nu,t) \in \Theta \times \Theta' \times \mathcal{T}} \|u_h(\mu, \nu, t)\| > 0$, $M = \operatorname{ess\,sup}_{(\mu,\nu,t) \in \Theta \times \Theta' \times \mathcal{T}} \|u_h(\mu, \nu, t)\| < \infty$,

2. $\mathcal{G}$ is Lipschitz-continuous with the constant $L > 0$.

A very important fact follows directly from the presumably allowed statement of such a well-defined map; the solution $u_h(\cdot, \cdot; \mu, \nu)$ exists and is unique, in at least an L^2 -embedded sense, since the time-derivate can be relaxed using a variational argument.

Assumption 4. Let $N \in \mathbb{N}$ be fixed. We assume that for s, s' sufficiently large, there are two maps $\psi_* : \mathbb{R}^n \rightarrow \mathbb{R}^N$ and $\psi'_* : \mathbb{R}^N \rightarrow \mathbb{R}^n$ which attain the perfect nonlinear embedding of the reduced solution manifold, as given in Definition (2.14) for any $n \leq 2(p + q) + 3$, i.e. for $\mathcal{S}_N := \{q(\mu, \nu, t) = \mathbb{A}^T u(\mu, \nu, t) \mid (\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}\}$, where $\mathbb{A} \in \mathbb{R}^{N_h \times N}$ is the POD matrix,

$$\delta_n(\mathcal{S}_N) = 0.$$

3.2 Projection-based Reduced Order Models

In this section we want to introduce some projection based methods for the dimensionality reduction of the finite dimensional parametric PDE. Such methods are all located in the (hierarchically) bigger group of *intrusive* ROMs, since they require one to explicitly know the parametric PDE at hand. Since we have embraced a very broad group of PDEs in the introduction, for this section, there will be some specification of the PDE needed.

All of the following methods will be discussed broadly, as the thesis concerns itself mostly with the later deep learning based ROMs.

For all of the methods, we remind ourselves of the starting situation: $\mathcal{H}_h \subset \mathcal{H}$ is a finite dimensional approximation space of our weak problem. As mentioned before, we also have employed discretized time-steps. We therefore have something along the lines of

$$\begin{cases} M(\mu, \nu) \hat{\partial}_t u_h^{\mu, \nu}(t) + A(\mu, \nu) u_h^{\mu, \nu}(t) + N(u_h^{\mu, \nu}(t), \mu, \nu) = f(t; \mu, \nu) & t \in \mathcal{T}, \\ u_h^{\mu, \nu}(0) = u_0(\mu, \nu) & t = 0, \end{cases} \quad (3.1)$$

for $\hat{\partial}_t$ being a discrete approximation of the time-derivative and $u_h^{\mu, \nu}(t) \in \mathbb{R}^{N_h}$ for $t \in [t_k, t_{k+1})$ and some $k = 0, \dots, N_t - 1$ the piecewise constant coefficient vector for the true solution

$$u_h(x, t; \mu, \nu) = \sum_{j=1}^{N_h} [u_h^{\mu, \nu}(t)]_j \varphi_j(x) \in \mathcal{H}_h, \text{ for each } t \in [t_k, t_{k+1}).$$

Definition 3.2 (Reduced Basis). Let $\mathcal{H}_n \subset \text{span}(u_h(\cdot, t; \mu, \nu) \in \mathcal{H}_h \mid (\mu, \nu, t) \in \mathcal{P}_1 \times \mathcal{P}_2 \times \mathcal{T})$ be an n -dimensional subspace of $\mathcal{H}_h$, then if $\{\hat{\varphi}_j\}_{j=1}^n$ is an orthonormalized basis of $\mathcal{H}_n$, we call this the *reduced basis*.

From this starting point, the goal of all *reduced basis methods* is to find a reduced basis $\{\hat{\varphi}_j\}_{j=1}^n$ and $\hat{u}_h^{\mu, \nu}(t) \in \mathbb{R}^n$ for $n \ll N_h$, such that

$$\hat{u}_h(x, t; \mu, \nu) = \sum_{j=1}^n [\hat{u}_h^{\mu, \nu}(t)]_j \hat{\varphi}_j(x) \in \mathcal{H}_n \subset \mathcal{H}_h,$$

where, in a yet vague formulation,

$$\hat{u}_h(\cdot, t; \mu, \nu) \approx u_h(\cdot, t; \mu, \nu) \quad \text{for all } \mu \in \Theta, \nu \in \Theta' \text{ and } t \in [0, T].$$

Two prominent examples of such reduced basis methods can be found in the following. Both will exhibit intrinsically an *offline* and an *online* part in the algorithm, whose separation we will mostly neglect. However, when remarking something about the offline procedure, we usually talk about the part of the algorithm that can be collected a priori to its purposed deployment. Here this is the collection and reduction of dimension using a reduced basis via samples. On the other hand, online refers to its counterpart; the many-query element of the procedure, such as the evaluation of the best-guess parameter-to-solution map. This terminology is used often throughout the literature as it instinctively makes sense in a software development environment.

3.2.1 Galerkin Proper Orthogonal Decomposition

As a prerequisite for Galerkin orthogonality, we assume here that the weak form giving rise to the equation at hand derives from an abstract bilinear form in the stationary part. This is justified if the non-linear summand is zero. The source for this subsection is (Quarteroni et al., 2015, p. 137).

The POD as introduced in Definition (2.10) is one of the most common methods for a reduction in a classical sense. We can use the system above to solve for the solution using a Galerkin projection for the following Ansatz:

$$\hat{u}_h(x, t; \mu, \nu) = \sum_{j=1}^n [\hat{u}_h^{\mu, \nu}(t)]_j \xi_j(x), \quad (3.2)$$

where ξ_j for $1 \leq j \leq n$ are the POD modes, as given by the construction of the POD, and $\hat{u}_h^{\mu, \nu}(t) \in \mathbb{R}^n$ is the coefficient vector, that solves the following reduced equation

$$\begin{cases} \mathbb{A}^T M(\mu, \nu) \mathbb{A} \hat{\partial}_t \hat{u}_h^{\mu, \nu}(t) + \mathbb{A}^T A(\mu, \nu) \mathbb{A} \hat{u}_h^{\mu, \nu}(t) = \mathbb{A}^T f(t; \mu, \nu) & t \in \mathcal{T}, \\ \hat{u}_h^{\mu, \nu}(0) = u_0(\mu, \nu) & t = 0. \end{cases} \quad (3.3)$$

The parameter-to-solution map can thus be recovered by the following algorithm.

Algorithm 2 POD-Galerkin

- 1: **Input:** $(\mu_j, \nu_j)_{j=1}^{N_s}$ data sample and $n < N_h$ reduced dimension
 - 2: **Output:** approximated parameter-to-solution-map $(\mu, \nu) \mapsto \hat{u}_h(\cdot, \cdot; \mu, \nu)$
 - 3: Compute $\{u_h^{\mu_j, \nu_j, t_{k_m}}\}_{m=1}^{N_t}$ from Equation (3.1) (with $N = 0$)
 - 4: $(\xi_j, \sigma_j)_{j=1}^n \leftarrow \text{POD}(u_h^{\mu_j, \nu_j, t_{k_m}}, n)$
 - 5: $\mathbb{A} \leftarrow \text{stack}(\xi_j, j = 1, \dots, n)$
 - 6: Build reduced Equation (3.3)
 - 7: For each (μ, ν) solve Equation (3.3)
 - 8: Recover $\hat{u}_h(\cdot, \cdot; \mu, \nu)$ via Equation (3.2)
-

Rework with regard to gram matrix

3.2.2 Greedy Proper Orthogonal Decomposition

An introduction to this method in the stationary and well-posed case can be found in (Siena et al., 2022). As done previously in the subsection above, we assume $N = 0$ in Equation (3.1). This gives us a coercive abstract bilinear form for the stationary part. Additionally let $\eta : \Theta \times \Theta' \rightarrow \mathbb{R}$ be a function estimating the error of projection under a specific parameter set, i.e. such that

$$\sup_{t \in \mathcal{T}} \|u_h(\mu, \nu, t) - \hat{u}_h(\mu, \nu, t)\| \leq \eta(\mu, \nu),$$

for $(\mu, \nu) \in \mathcal{P}$. Purely using POD-Galerkin can encounter major pitfalls, namely if the range of $\mathcal{P}$ is extensive, since then calculating a singular value decomposition for the correlation matrix might get too cumbersome to provide all areas of interest in the set. Moreover, to obtain the snapshots for fixed parameters, a complete time-trajectory must be computed, which entails a lot of overhead of otherwise useless information. To overcome this (Grepl & Patera, 2005) proposed, and later (Haasdonk & Ohlberger, 2008) improved upon, the mixed POD-Greedy approach, which takes a POD over the time-trajectories (or, in the improved version, the error trajectories) while using the Greedy approach to handle the parameter samples. It follows a similar workflow, as the POD-Galerkin, since the only thing, that changes is the choice of basis, and the fact that the size of the basis is chosen according to the tolerance in the offline part of the algorithm, i.e.

$$\hat{u}_h(x, t; \mu, \nu) = \sum_{j=1}^N [\hat{u}_h^{\mu, \nu}(t)]_j \xi_j(x), \quad (3.4)$$

identifies the reduced solution of dimension N , where ξ_j are the basis functions for $1 \leq j \leq N$ and the coefficient vector being the solution to

$$\begin{cases} \mathbb{V}_{\text{rb}}^T M(\mu, \nu) \mathbb{V}_{\text{rb}} \hat{\partial}_t \hat{u}_h^{\mu, \nu}(t) + \mathbb{V}_{\text{rb}}^T A(\mu, \nu) \mathbb{V}_{\text{rb}} \hat{u}_h^{\mu, \nu}(t) = \mathbb{V}_{\text{rb}}^T f(t; \mu, \nu) & t \in \mathcal{T}, \\ \hat{u}_h^{\mu, \nu}(0) = u_0(\mu, \nu) & t = 0. \end{cases} \quad (3.5)$$

The complexity of the offline computation for this algorithm is additive, and not multiplicative, in the number of training data $\mathcal{P}$, which is a huge advantage when compared to the direct POD-Galerkin approach.

Algorithm 3 POD-Greedy-Galerkin

- 1: **Input:** tol , $\mathcal{Z} = 0$, $N = 0$, $(\mu_j, \nu_j)_{j=1}^{N_s}$, N_1 and N_2
 - 2: **Output:** approximated parameter-to-solution-map $(\mu, \nu) \mapsto \hat{u}_h(\cdot, \cdot; \mu, \nu)$
 - 3: **for** $j = 1, \dots, N_s$ **do**
 - 4: Compute $\{u_h^{\mu_j, \nu_j, t_{k_m}}\}_{m=1}^{N_t}$ from Equation (3.1) (with $N = 0$)
 - 5: $(\zeta_j, \lambda_j)_{j=1}^{N_1} \leftarrow \text{POD}(\{u_h^{\mu_j, \nu_j, t_{k_m}}\}_{m=1}^{N_t}, N_1)$
 - 6: $\mathcal{Z} \leftarrow \{\mathcal{Z}, \{\zeta_1, \dots, \zeta_{N_1}\}\}$
 - 7: $N \leftarrow N + N_2$
 - 8: Compress: $(\xi_j, \sigma_j)_{j=1}^n \leftarrow \text{POD}(\mathcal{Z}, N)$
 - 9: $(\mu_{j+1}, \nu_{j+1}) \leftarrow \arg \max_{(\mu, \nu) \in \mathcal{P}} \eta(\mu, \nu)$
 - 10: **if** $\eta(\mu_{j+1}, \nu_{j+1}) > \text{tol}$ **then**
 - 11: $j \leftarrow j + 1$
 - 12: **else**
 - 13: **break**
 - 14: **end if**
 - 15: **end for**
 - 16: $\mathbb{V}_{\text{rb}} \leftarrow \text{stack}(\xi_1, \dots, \xi_N)$
 - 17: Build reduced Equation (3.5)
 - 18: For each (μ, ν) solve Equation (3.5)
 - 19: Recover $\hat{u}_h(\cdot, \cdot; \mu, \nu)$ via Equation (3.4)
-

Remark 3.3. It would be possible to include the physical parameter into the first POD scheme in each iteration, which could give a high yield for this specific problem as the Kolmogorov n -width decay as imposed by Assumption (1) implies. However since we lose generality by setting $N = 0$ in both projection-based algorithm, this is only an argument for divergent discussions.

3.3 Data-driven Reduced Order Models

Introduce POD-DL-ROM

A general Deep learning based reduced order model (DL-ROM) firstly appeared in architecture and training in the paper (Fresca et al., 2020); to properly approximate the potentially non-linear solution manifold of the high fidelity FOM, we employ an Autoencoder (AE), and on the reduced manifold we employ some other Deep Learning architecture, like the proposed Deep Feed-Forward

Neural Network (DFNN), to learn the latent dynamics of the parametric PDE. This concludes in the following approximation of the FOM solution u_h

$$\hat{u}_h(t; \mu, \nu) = \Psi_h(\Phi_n(t; \mu, \nu, \theta_{DF}), \theta_D) \in \mathbb{R}^{N_h}, \quad (3.1)$$

where we set $\Psi_h = f_h^D : \mathbb{R}^n \times \Theta_D \rightarrow \mathbb{R}^{N_h}$ for the decoder (and $f_n^E : \mathbb{R}^{N_h} \times \Theta_E \rightarrow \mathbb{R}^n$ the corresponding encoder) of a convolutional AE. Then f_n^E denotes the projection onto the reduced trial solution manifold and we assume $\Phi_n = \Phi_n^{DF} : \Gamma \times \Theta \times \Theta' \times \Theta_{DF} \rightarrow \mathbb{R}^n$ to be the DFNN mapping the dynamics of the parametric PDE onto the reduced trial solution manifold.

The subsequent training is then performed via a loss function of a convex combination of both, the projection error using the encoder and the dynamics error considering both the decoder and the feed forward network, i.e. for $\theta = (\theta_D, \theta_E, \theta_{DF})$ define the optimal weights via

$$\theta^* = \arg \min_{\theta} \frac{1}{N_s} \sum_{i,j=1}^{N_{train}} \sum_{k=1}^{N_t} \frac{\omega_h}{2} \|u_h(t^k; \mu_i, \nu_j) - f_h^D(\Phi_n^{DF}(t^k; \mu_i, \nu_j, \theta_{DF}), \theta_D), \theta_D)\|^2 \quad (3.2)$$

$$+ \frac{1 - \omega_h}{2} \|f_n^E(u_h(t^k; \mu_i, \nu_j), \theta_E) - \Phi_n^{DF}(t^k; \mu_i, \nu_j, \theta_{DF})\|^2. \quad (3.3)$$

3.3.1 Deep Orthogonal Decomposition based Reduction

Introduce both DOD-DL-ROM and DOD+DNN

For comparison, one can see, that the POD-DL-ROM works significantly more efficient than the DL-ROM by pre-reducing the dimension N_h to some $N < N_h$ in a linear POD-based fashion. This has the disadvantage of potentially losing fidelity, if the Kolmogorov N -width is rather large, which is the case for non-linear PDEs, relying on geometric circumstances. For this, we might instead of using snapshot-based rPOD for the matrix S (optionally the correlation matrix) rather directly employ a Deep Orthogonal Decomposition to find the reduction matrix in dependency of $\mu \in \Theta$ and $t \in \Gamma$.

Thus we have a common denominator for the DOD-DL-ROM approach. Defining $V : \Gamma \times \Theta \times \Theta_{DOD} \rightarrow \mathbb{R}^{N_h \times N}$; $V(t; \mu, \theta_{DOD}) \mapsto \mathbb{A} \tilde{V}_{N_A}(t; \mu, \theta_{DOD})$, where $\mathbb{A} \in \mathbb{R}^{N_h \times N_A}$ is the pre-reduction POD for $N < N_A < N_h$ employed to deal with exceptionally large inherent dimensions N_h and $\tilde{V}_{N_A}(t; \mu, \theta_{DOD}) \in \mathbb{R}^{N_A \times N}$ is called *inner module* of the DOD, yields the foundation of the reduction consideration. From hereon, there are a couple of different approaches, which lead to similar ideas; one is able to, after learning the DOD-map, either suppose, the remaining reduced solution manifold is "linear" enough to work directly with a coefficient finder network (Option A) or a further non-linear reduction, potentially by an Autoencoder, is needed (Option B). As an additional, maybe more ambiguous regarding error estimates, we want to introduce a CoLoRA-based ROM, which first tries to use the DOD-NN reductor for the stationary equivalent of the problem (if existent) and then employs a CoLoRA architecture to reduce the dimension up to the dynamics of the system (Option C).

For Options A and B: To use the same loss function as in the (POD-)DL-ROM, we do need to guarantee a previous construction of the inner module of the DOD. This first step is done with a separate training with a time dependent analogue to the DOD, i.e.

$$\theta_{DOD}^* = \arg \min_{\theta_{DOD}} \frac{1}{N_s} \sum_{i,j=1}^{N_{train}} \sum_{k=1}^{N_t} \|u_h(t^k; \mu_i, \nu_j) - V(t^k; \mu_i, \theta_{DOD}) V^T(t^k; \mu_i, \theta_{DOD}) \mathbb{G} u_h(t^k; \mu_i, \nu_j)\|^2. \quad (3.4)$$

3.3.2 DOD+DNN

Let $\Phi_n^{DLNN} : \Gamma \times \Theta \times \Theta' \times \Theta_{DLNN} \rightarrow \mathbb{R}^N$ be some Deep Feed-Forward Neural Network, then the overall workflow can be summed up by

$$\hat{u}_h(t; \mu, \nu, \theta_{DOD}, \theta_{DFNN}) = \mathbb{A} \tilde{V}_{N_A}(t; \mu, \theta_{DOD}) \cdot \Phi_n^{DFNN}(t; \mu, \nu, \theta_{DFNN}). \quad (3.5)$$

This is a rather obvious extension of (Franco et al., 2024), but works surprisingly well. It will be the basic benchmark to overcome for the following algorithms. As for training a general comparison between the projection of a true solution and the prediction of the Feed Forward Network works well, i.e. for $\theta = \theta_{DFNN}$ let the optimal weights be given by

$$\theta^* = \arg \min_{\theta} \frac{1}{N_s} \sum_{i,j=1}^{N_{train}} \sum_{k=1}^{N_t} \| V^T(t^k; \mu_i, \theta_{DOD}^*) u_h(t^k; \mu_i, \nu_j) - \Phi_n^{DFNN}(t^k; \mu_i, \nu_j, \theta_{DFNN}) \|. \quad (3.6)$$

3.3.3 DOD-DL-ROM

Let f_N^D and f_n^E be the decoder and encoder of an AE respectively as above. Each of these non-linear DOD-DL-ROMs is subject to further dimensional reduction and thus is not comparable directly to the linear method above. Nonetheless in terms of reduction in dimensionality both methods can be comparable, if the latent dynamics dimension n is the same in both cases. The general workflow is given by

$$\hat{u}_h(t; \mu, \nu, \theta_{DOD}, \theta_D, \theta_{DFNN}) = \mathbb{A} \tilde{V}_{N_A}(t; \mu, \theta_{DOD}) \cdot f_N^D(\Phi_n^{DFNN}(t; \mu, \nu, \theta_{DFNN}), \theta_D). \quad (3.7)$$

Since for this, we employ an AE, we optimize for two loss functions, since the employed decoder is tied strictly to its respective encoder being able to correctly project onto the low-dimensional subspace of the latent dynamics of the PDE. For $\theta = (\theta_D, \theta_E, \theta_{DFNN})$ define the optimal weights via

$$\begin{aligned} \theta^* = \arg \min_{\theta} \frac{1}{N_s} \sum_{i,j=1}^{N_{train}} \sum_{k=1}^{N_t} & \frac{\omega_h}{2} \| u_h(t^k; \mu_i, \nu_j) - V(t^k; \mu_i, \theta_{DOD}^*) \cdot f_h^D(\Phi_n^{DFNN}(t^k; \mu_i, \nu_j, \theta_{DFNN}), \theta_D), \theta_D) \|^2 \\ & + \frac{1 - \omega_h}{2} \| V^T(t^k; \mu_i, \theta_{DOD}^*) \cdot f_n^E(u_h(t^k; \mu_i, \nu_j), \theta_E) - \Phi_n^{DFNN}(t^k; \mu_i, \nu_j, \theta_{DFNN}) \|^2. \end{aligned}$$

3.3.4 CoLoRA-DL-ROM

Introduce CoLoRA DL-ROM

Here we aim at minimizing the dimension through two major reductions; the first being the reduction of the stationary DOD to the latent dimension of the μ -invariant solution manifold of the stationary equivalent to the problem (regarding the Dirichlet boundary conditions as the initial conditions on some a priori defined boundary), and the second concerning the actual reduction with CoLoRA-layered architecture to the latent dynamic dimension $n \ll N_h$. The general workflow is thus given by

$$\hat{u}_h(t; \mu, \nu, \theta_{sDOD}^*, \theta_C) = \mathbb{A} \cdot \mathcal{C}_L(\dots (\mathcal{C}_2(\mathcal{C}_1(V_0(\mu, \theta_{sDOD}^*) \cdot \phi_0(\mu, \nu, \theta_{sDOD}^*)), \theta_{C_1}), \theta_{C_2}), \theta_{C_L}), \quad (3.8)$$

where $\mathcal{C}_i : \mathbb{R}^{N_A} \times \Theta_C \rightarrow \mathbb{R}^{N_A}$; $\mathcal{C}_i(x, \theta_{C_i}) = W_i x + A_i \text{diag}(\alpha_i(\nu, t)) B_i x + b_i$ for all $1 \leq i \leq L$. Remark, that we will still employ a pre-reduction using the rPOD matrix $\mathbb{A}$ stemming from the same snapshot matrix S as in both Options before.

Chapter 4

Approximation Theory for Reduced Order Models

Include proof and discussion for general DL-ROMs

Proposition 4.1. Let $f \in L^2(\Theta \times \Theta' \times \mathcal{T})$. Under Assumption (2), it is true, that

$$\mathbb{E} \left[\left| \int_{\Theta \times \Theta' \times \mathcal{T}} f(\mu, \nu, t) d(\mu, \nu, t) - \frac{|\Theta \times \Theta' \times \mathcal{T}|}{N_{data}} \sum_{i=1}^{N_{s1}} \sum_{j=1}^{N_{s2}} \sum_{k=1}^{N_t} f(\mu_i, \nu_j, t_k) \right| \right] \leq \mathcal{O}(N_s^{-1/2} + N_t^{-1}).$$

4.1 Relative Error of POD+DNNs

Discuss relative errors for the plain POD+DNN

Theorem 4.2. Let $\mathcal{G} : (\mu, \nu, t) \mapsto u_h(\mu, \nu, t)$ for all $\mu \in \Theta, \nu \in \Theta'$ and $t \in \mathcal{T}$ be the parameter-to-solution map. Here we consider a general POD+DNN structure, thus neglecting the potentially different nature of Θ and Θ' , i.e. $\mathcal{G}(\mu, \nu, t) \approx V\hat{q}$, where $\hat{q} : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^N$ is a neural network trained using data $(\mu_i, \nu_i, t_i)_{i=1}^{N_{data}}$ and V is the POD matrix, as described in (2.10). Then under the Assumptions (2) and (3), we have

$$\mathcal{E}_R \leq \mathcal{E}_S + \mathcal{E}_{POD} + \mathcal{E}_{NN}, \quad (4.1)$$

where

1. The sampling error

$$\mathcal{E}_S = \mathcal{E}_S \left(\mathcal{G}, (\mu_i, \nu_i, \tau_i)_{i=1}^{N_{data}}, N \right)$$

satisfies $\mathcal{E}_S \rightarrow 0$ as $N_s, N_t \rightarrow \infty$, with expectation

$$\mathbb{E}[\mathcal{E}_S] = \mathcal{O}(N_s^{-1/4} + N_t^{-1}).$$

2. The POD projection error

$$\mathcal{E}_{POD} = \mathcal{E}_{POD} \left(\mathcal{G}, (\mu_i, \nu_i, \tau_i)_{i=1}^{N_{data}}, N \right)$$

satisfies $\mathcal{E}_{POD} \rightarrow \mathcal{E}_{POD,\infty}$ almost surely as $N_s, N_t \rightarrow \infty$, where

$$\mathcal{E}_{POD,\infty} = \mathcal{E}_{POD,\infty}(\mathcal{G}, N)$$

is independent of the data snapshots and proportional to the linear KnW of the manifold $\mathcal{S} = \{u(\mu, \nu, t) \mid (\mu, \nu, t) \in \Theta' \times \mathcal{T}\}$, i.e. $d_N(\mathcal{S})$.

3. The neural network approximation error

$$\mathcal{E}_{NN} = \mathcal{E}_{NN}(\mathcal{G}, N, \hat{q})$$

is arbitrarily small depending on the complexity and structure of the neural network $\hat{q}$.

Proof. With the knowledge of $\|\cdot\|_{L_\omega^2}$ being a norm, we know it must be subject to the triangle inequality, thus

$$\mathcal{E}_R = \left(\int_{\Theta \times \Theta' \times \mathcal{T}} \frac{\|u(\mu, \nu, t) - V\hat{q}(\mu, \nu, t)\|}{\|u(\mu, \nu, t)\|} d(\mu, \nu, t) \right)^{1/2} \quad (4.2)$$

$$\leq \|u(\mu, \nu, t) - VV^T u(\mu, \nu, t)\|_{L_\omega^2} + \|VV^T u(\mu, \nu, t) - V\hat{q}(\mu, \nu, t)\|_{L_\omega^2}. \quad (4.3)$$

Let $q(\mu, \nu, t) := V^T u(\mu, \nu, t)$ be a reduced solution according to the POD basis. Then the natural definition of the neural network approximation error is

$$\mathcal{E}_{NN} = \left(\int_{\Theta \times \Theta' \times \mathcal{T}} \frac{\|Vq(\mu, \nu, t) - V\hat{q}(\mu, \nu, t)\|^2}{\|u(\mu, \nu, t)\|^2} d(\mu, \nu, t) \right)^{1/2}. \quad (4.4)$$

This marks the second part in (4.3). This is only dependent on the ability of the neural network to identifying the best choice on a given linear sub-manifold of the solution manifold. The first part is concerned with the error generated from the best *possible* approximation of such a linear sub-manifold.

For this, we can bound the norm $\|\cdot\|_{L_\omega^2} \leq m^{-1}\|\cdot\|$, where $m > 0$ is the constant stemming from Assumption (3). Further we remind ourselves of the definition of the discrete correlation matrix $K = |\Theta \times \Theta \times \mathcal{T}| N_{data}^{-1} U U^T$ and its eigenvalues σ_k in (2.10). Using $\sqrt{a+b} \leq \sqrt{a} + \sqrt{b}$ for all $a, b \geq 0$, we get

$$\begin{aligned} & \|u(\mu, \nu, t) - VV^T u(\mu, \nu, t)\|_{L_\omega^2} \\ & \leq m^{-1} \left(\int_{\Theta \times \Theta' \times \mathcal{T}} \|u(\mu, \nu, t) - VV^T u(\mu, \nu, t)\|^2 d(\mu, \nu, t) \right)^{1/2} \\ & \leq m^{-1} \left(\int_{\Theta \times \Theta' \times \mathcal{T}} \|u(\mu, \nu, t) - VV^T u(\mu, \nu, t)\|^2 d(\mu, \nu, t) - \sum_{k>N} \sigma_k^2 + \sum_{k>N} \sigma_k^2 \right)^{1/2} \\ & \leq m^{-1} \left(\left| \int_{\Theta \times \Theta' \times \mathcal{T}} \|u(\mu, \nu, t) - VV^T u(\mu, \nu, t)\|^2 d(\mu, \nu, t) - \sum_{k>N} \sigma_k^2 \right| + \sum_{k>N} \sigma_k^2 \right)^{1/2} \\ & \leq m^{-1} \underbrace{\left| \int_{\Theta \times \Theta' \times \mathcal{T}} \|u(\mu, \nu, t) - VV^T u(\mu, \nu, t)\|^2 d(\mu, \nu, t) - \sum_{k>N} \sigma_k^2 \right|^{1/2}}_{=: \mathcal{E}_S} + m^{-1} \underbrace{\sqrt{\sum_{k>N} \sigma_k^2}}_{=: \mathcal{E}_{POD}}. \end{aligned}$$

This concludes the estimate up to the facts about the convergence of the errors. Recalling (2.12), we can rewrite the sampling error as follows

$$\mathcal{E}_S = m^{-1} \left| \int_{\Theta \times \Theta' \times \mathcal{T}} \|u(\mu, \nu, t) - VV^T u(\mu, \nu, t)\|^2 d(\mu, \nu, t) - \frac{|\Theta \times \Theta' \times \mathcal{T}|}{N_{data}} \sum_{j=1}^{N_{data}} \|u_j - VV^T u_j\|^2 \right|^{1/2}$$

Furthermore, we define using the compactness postulated in Assumption (2) and the boundedness in Assumption (3), the L^2 -error map via

$$f(\mu, \nu, t) := \|u(\mu, \nu, t) - VV^T u(\mu, \nu, t)\|^2 \leq M^2 \|I - VV^T\|^2 < \infty,$$

where we justify neglecting the dependence on u in the mapping, since $\mathcal{G}(\mu, \nu, t)$ is assumed to be known and can be projected onto the N_h -dimensional solution manifold. Thus with $f \in L^2(\Theta \times \Theta' \times \mathcal{T})$, we know that by Proposition (4.1), $\mathbb{E}[\mathcal{E}_S] \leq \mathcal{O}(N_s^{-1/4} + N_t^{-1/2})$.

By the strong law of large numbers, we can thus conclude, that $\mathcal{E}_S \rightarrow 0$ almost surely, as $N_t, N_s \rightarrow \infty$, and further by (2.11), we have that $\mathcal{E}_{POD} \rightarrow \mathcal{E}_{POD, \infty}$ as $N_t, N_s \rightarrow \infty$, where

$$\mathcal{E}_{POD, \infty} = m^{-1} \sqrt{\sum_{k>N} \sigma_{k, \infty}^2}. \quad (4.5)$$

□

Theorem 4.3. *Under the assumptions of Theorem (4.2), we have that*

$$\mathcal{E}_R \geq \frac{m}{M} \mathcal{E}_{DOD, \infty},$$

where $\mathcal{E}_{DOD, \infty} := m^{-1} \sqrt{\sum_{k>N} \sigma_{k, \infty}^2}$.

Proof.

Proof this

□

4.2 Relative Error of DOD+DNNs

Extrapolate to DOD+DNN

Lemma 4.4 (Existence of the DOD-Module).

Rework the proof and statement according to the setting

Let $\Theta \subset \mathbb{R}^p$ and $\Theta' \subset \mathbb{R}^q$ be two compact sets. Let

$$\mathcal{F} : \Theta \times \Theta' \rightarrow \mathbb{R}^{N_h}; (\mu, \nu) \mapsto \mathbf{u}_{\mu, \nu}$$

be continuous. For each $\mu \in \Theta$, let $\mathcal{S}_\mu := \{\mathbf{u}_{\mu, \nu}\}_{\nu \in \Theta'} \subset \mathcal{F}(\Theta \times \Theta')$ be the μ -submanifold in the image of $\mathcal{F}$. Let $\mathbb{G}$ be the Gram matrix associated with an inner product in $\mathbb{R}^{N_h}$, and let $\|\cdot\|$ be its

corresponding norm. Then, for every $\varepsilon > 0$ there is a ReLU matrix-valued deep neural network $\mathbf{V} : \mathbb{R}^p \rightarrow \mathbb{R}^{N_h \times n}$ such that

$$\mathbb{E}_{\mu, \nu} \|\mathbf{u}_{\mu, \nu} - \mathbf{V}_\mu \mathbf{V}_\mu^T \mathbb{G} \mathbf{u}_{\mu, \nu}\| < \varepsilon + \mathbb{E}_\mu[d_n(\mathcal{S}_\mu)]$$

where $\mathbf{V}_\mu := \mathbf{V}(\mu)$.

Proof. Let $n \in \mathbb{N}$ be fixed. Assuming we have proven, that $|\mathbb{V}_{ij}| \leq 1$ w.l.o.g., we define $J : \Theta \times [-1, 1]^{N_h \times n} \rightarrow \mathbb{R}$ as

$$J(\mu, \mathbb{V}) := \mathbb{E}_\nu \|\mathbf{u}_{\mu, \nu} - \mathbb{V} \mathbb{V}^T \mathbb{G} \mathbf{u}_{\mu, \nu}\|, \quad (4.1)$$

and let some $\varepsilon > 0$. Then the continuity of J implies uniform continuity by the compactness of its pre-image, i.e. there is $\delta > 0$ such that

$$|\mu - \mu'| + \|\mathbb{V} - \mathbb{V}'\|_\infty < \delta \implies |J(\mu, \mathbb{V}) - J(\mu', \mathbb{V}')| < \varepsilon.$$

Then there is a Borel measurable map $s : \Theta \rightarrow [-1, 1]^{N_h \times n}$ given by

$$s : \mu \mapsto \operatorname{argmin}_{\mathbb{V} \in [-1, 1]^{N_h \times n}} J(\mu, \mathbb{V}).$$

This map is bounded (given by continuity of J and compactness of its image) and hence $s \in L^1(\Theta; \mathbb{R}^{N_h \times n})$, i.e. $\|s\|_{L^1(\Theta; \mathbb{R}^{N_h \times n})} := \int_\Theta \|s(\mu)\|_\infty d\mu(\mu) < \infty$. This means that by Hornik's Theorem (Hornik, 1991) there is a deep ReLU neural network $\mathbf{V}_0 : \Theta \rightarrow \mathbb{R}^{N_h \times n}$ such that $\mathbb{E}_\mu |\mathbf{V}_0(\mu) - s(\mu)| < \delta$ for all $\delta > 0$. Now let ρ be a entry-wise ReLU activation function and we consider a three-layered network $L : \mathbb{R}^{N_h \times n} \rightarrow \mathbb{R}^{N_h \times n}$ given by

$$L(\mathbf{x}) := \mathbf{e} - \mathbb{I}_{N_h \times n} \rho(-\mathbb{I}_{N_h \times n} \rho(\mathbb{I}_{N_h \times n} \mathbf{x} + \mathbf{e}) + 2\mathbf{e}),$$

where $\mathbb{I}_{N_h \times n}$ is the identity tensor on $\mathbb{R}^{N_h \times n}$, and $\mathbf{e}$ is a $N_h \times n$ matrix with ones as entries. Thus for each entry $x_{i,j}$ of $\mathbf{x}$, we have $L(x_{i,j}) = 1 - \rho(2 - \rho(x_{i,j} + 1)) \in [-1, 1]$. Thus, since L is 1-Lipschitz and L only effects values outside $[-1, 1]$, and hence $L \circ s = s$, we have for $\mathbf{V}(\mu) := L \circ \mathbf{V}_0(\mu)$

$$\mathbb{E} |\mathbf{V}(\mu) - s(\mu)| = \mathbb{E} |L \circ \mathbf{V}_0(\mu) - L \circ s(\mu)| \leq \mathbb{E} |\mathbf{V}_0(\mu) - s(\mu)| < \delta.$$

With this we can take $\mathbf{V}(\mu)$ to be in the pre-image of J and therefore conforming to its uniform continuity, i.e. take $\varepsilon > 0$, s.t.

$$|J(\mu, s(\mu)) - J(\mu, \mathbf{V}(\mu))| < \varepsilon.$$

In total we calculate with $\mathbb{E}_{\mu, \nu} \|\mathbf{u}_{\mu, \nu} - \mathbf{V}_\mu \mathbf{V}_\mu^T \mathbb{G} \mathbf{u}_{\mu, \nu}\| = \mathbb{E}_\mu [J(\mu, \mathbf{V}(\mu))]$,

$$\begin{aligned} \mathbb{E}_{\mu, \nu} \|\mathbf{u}_{\mu, \nu} - \mathbf{V}_\mu \mathbf{V}_\mu^T \mathbb{G} \mathbf{u}_{\mu, \nu}\| &\leq \mathbb{E}_\mu [J(\mu, \mathbf{V}(\mu)) - J(\mu, s(\mu))] + \mathbb{E}_\mu [J(\mu, s(\mu))] \\ &< \varepsilon + \mathbb{E}_\mu \left[\min_{\mathbb{V}} J(\mu, \mathbb{V}) \right] \\ &\leq \varepsilon + \mathbb{E}_\mu \left[\min_{\mathbb{V}} \sup_{\nu} \|\mathbf{u}_{\mu, \nu} - \mathbb{V} \mathbb{V}^T \mathbb{G} \mathbf{u}_{\mu, \nu}\| \right] \end{aligned}$$

□

Theorem 4.5. Let $\mathcal{G} : (\mu, \nu, t) \mapsto u_h(\mu, \nu, t)$ for all $\mu \in \Theta, \nu \in \Theta'$ and $t \in \mathcal{T}$ be the parameter-to-solution map. Here we consider a general DOD+DNN structure, i.e. $\mathcal{G}(\mu, \nu, t) \approx V(\mu, t)\hat{q}(\mu, \nu, t)$, where $\hat{q} : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^N$ is a neural network trained using data $(\mu_i, \nu_i, t_i)_{i=1}^{N_{data}}$ and $V : \mathbb{R}^{p+1} \rightarrow \mathbb{R}^{N_h \times N}$ is the DOD neural network, as described in (3.3.2). Then under the Assumptions (2) and (3), we have

$$\mathcal{E}_R \leq \mathcal{E}_S + \mathcal{E}_{DOD} + \mathcal{E}_{NN}, \quad (4.2)$$

where

1. The sampling error

$$\mathcal{E}_S = \mathcal{E}_S(\mathcal{G}, (\mu_i, \nu_i, \tau_i)_{i=1}^{N_{data}}, N)$$

satisfies $\mathcal{E}_S \rightarrow 0$ as $N_s, N_t \rightarrow \infty$, with expectation

$$\mathbb{E}[\mathcal{E}_S] = \mathcal{O}(N_s^{-1/4} + N_t^{-1}).$$

2. The DOD projection error

$$\mathcal{E}_{DOD} = \mathcal{E}_{DOD}(\mathcal{G}, (\mu_i, \nu_i, \tau_i)_{i=1}^{N_{data}}, N)$$

satisfies $\mathcal{E}_{DOD} \rightarrow \mathcal{E}_{DOD,\infty}$ almost surely as $N_s, N_t \rightarrow \infty$, where

$$\mathcal{E}_{DOD,\infty} = \mathcal{E}_{DOD,\infty}(\mathcal{G}, N)$$

is independent of the data snapshots and arbitrarily close, with regards to the capabilities of the DOD neural network, to the expected linear KnW of the submanifold $\mathcal{S}_{\mu,t} = \{u(\mu, \nu, t) \mid \nu \in \Theta'\}$ up to a constant, i.e. $\mathbb{E}_{\mu,t}[d_N(\mathcal{S}_{\mu,t})]$.

3. The neural network approximation error

$$\mathcal{E}_{NN} = \mathcal{E}_{NN}(\mathcal{G}, N, \hat{q})$$

satisfies $\mathcal{E}_{NN} \rightarrow d_N(\mathcal{S}_\mu)$ in expectation, as $N_s, N_t \rightarrow \infty$.

Proof. In the core this proof is the same as the one of Theorem (4.2), we will therefore only consider the parts in which they are not interchangeable; the use of the triangle inequality gives us the neural network error in the same way, with only the additional factor being dependent on the choice and sophistication of the DOD map,

$$\mathcal{E}_{NN} = \left(\int_{\Theta \times \Theta' \times \mathcal{T}} \frac{\|V(\mu, t)q(\mu, \nu, t) - V(\mu, t)\hat{q}(\mu, \nu, t)\|^2}{\|u(\mu, \nu, t)\|^2} d(\mu, \nu, t) \right)^{1/2}. \quad (4.3)$$

Moreover, we can extrapolate the same procedure of splitting the remaining term into the respective errors

$$\|u(\mu, \nu, t) - V(\mu, t)V(\mu, t)^T u(\mu, \nu, t)\|_{L_w^2} \leq \mathcal{E}_S + \mathcal{E}_{DOD}, \quad (4.4)$$

where

$$\mathcal{E}_S = m^{-1} \left| \int_{\Theta \times \Theta' \times \mathcal{T}} \|u(\mu, \nu, t) - V(\mu, t)V(\mu, t)^T u(\mu, \nu, t)\|^2 d(\mu, \nu, t) \right. \\ \left. - \frac{|\Theta \times \Theta' \times \mathcal{T}|}{N_{data}} \sum_{i=1}^{N_{s1}} \sum_{j=1}^{N_{s2}} \sum_{k=1}^{N_t} \|u(\mu_i, \nu_j, t_k) - V(\mu_i, t_k)V(\mu_i, t_k)^T u(\mu_i, \nu_j, t_k)\|^2 \right|^{1/2}$$

and

$$\mathcal{E}_{DOD} = \frac{m^{-1}|\Theta \times \Theta' \times \mathcal{T}|}{N_{data}} \left| \sum_{i=1}^{N_{s1}} \sum_{j=1}^{N_{s2}} \sum_{k=1}^{N_t} \|u(\mu_i, \nu_j, t_k) - V(\mu_i, t_k)V(\mu_i, t_k)^T u(\mu_i, \nu_j, t_k)\|^2 \right|^{1/2}.$$

This leaves us with the proof for the convergence statements. While $\mathcal{E}_S \rightarrow 0$ as $N_s, N_t \rightarrow \infty$ can be shown in the same manner as it has been in aforementioned proof above, the DOD error is more tricky.

Using the triangle inequality again and combining it with the findings of Lemma (4.4) we get for any $\varepsilon > 0$ and a suitable amount of complexity of the ReLU neural network $V : \mathbb{R}^{p+1} \rightarrow \mathbb{R}^{N_h \times N}$

$$\begin{aligned} \mathcal{E}_{DOD} &\leq m^{-1} \left| \int_{\Theta \times \Theta' \times \mathcal{T}} \|u(\mu, \nu, t) - V(\mu, t)V(\mu, t)^T u(\mu, \nu, t)\|^2 d(\mu, \nu, t) \right|^{1/2} \\ &\quad + \mathcal{E}_{DOD} - m^{-1} \left| \int_{\Theta \times \Theta' \times \mathcal{T}} \|u(\mu, \nu, t) - V(\mu, t)V(\mu, t)^T u(\mu, \nu, t)\|^2 d(\mu, \nu, t) \right|^{1/2} \\ &\xrightarrow{N_s, N_t \rightarrow \infty} m^{-1} \left| \int_{\Theta \times \Theta' \times \mathcal{T}} \|u(\mu, \nu, t) - V(\mu, t)V(\mu, t)^T u(\mu, \nu, t)\|^2 d(\mu, \nu, t) \right|^{1/2} \\ &< m^{-1}\varepsilon + m^{-1} \int_{\Theta \times \mathcal{T}} d_N(\mathcal{S}_{\mu, t}) d(\mu, t) \end{aligned}$$

□

Lemma 4.6 (Projection Error). Let N_{s2} be given under Assumption (2), then for all $\varepsilon > 0$ there is a ReLU neural network $V : \mathbb{R}^{p+1} \rightarrow \mathbb{R}^{N_h \times N}$, such that for $u_j := u(\mu, \nu_j, t)$ with given $\mu \in \Theta$ and $t \in \mathcal{T}$

$$\sup_{(\mu, t) \in \Theta \times \mathcal{T}} \left| \frac{1}{N_{s2}} \sum_{j=1}^{N_{s2}} \|u_j - V(\mu, t)V(\mu, t)^T u_j\|^2 - \min_{W \in \mathbb{R}^{N_h \times N}} \frac{1}{N_{s2}} \sum_{j=1}^{N_{s2}} \|u_j - WW^T u_j\|^2 \right|^{1/2} < \varepsilon.$$

Proof. The existence and identity of V^* s.t.

$$V^* = \operatorname{argmin}_{W \in \mathbb{R}^{N_h \times N}} \sum_{j=1}^{N_{s2}} \|u_j - V^*(V^*)^T u_j\|^2$$

is given via the statement in Proposition (2.12) with the POD basis. Hence defining a norm via

$$\|(u_j)_{j=1}^{N_{s2}}\|_{train} := \left| \sum_{j=1}^{N_{s2}} \|u_j\|^2 \right|^{1/2}$$

gives us the following inequality firstly using the monotonicity of the square root and then using the triangle inequalities multiple times

$$\begin{aligned} & N_{s_2}^{-1/2} \sup_{(\mu, t) \in \Theta \times \mathcal{T}} \left| \sum_{j=1}^{N_{s_2}} \|u_j - V(\mu, t)V(\mu, t)^T u_j\|^2 - \sum_{j=1}^{N_{s_2}} \|u_j - V^*(V^*)^T u_j\|^2 \right|^{1/2} \\ & \leq N_{s_2}^{-1/2} \sup_{(\mu, t) \in \Theta \times \mathcal{T}} \left| \sum_{j=1}^{N_{s_2}} \|V(\mu, t)V(\mu, t)^T u_j - V^*(V^*)^T u_j\|^2 \right|^{1/2}, \end{aligned}$$

then define $q_j := (V^*)^T u_j$ and $\hat{q}_j := V(\mu, t)^T u_j$ and their training vectors accordingly, i.e. $q := (q_j)_{j=1}^{N_{s_2}}, \hat{q} := (\hat{q}_j)_{j=1}^{N_{s_2}}$

$$\begin{aligned} & \leq N_{s_2}^{-1/2} \sup_{(\mu, t) \in \Theta \times \mathcal{T}} \|V(\mu, t)\hat{q} - V(\mu, t)q\|_{train} + \|V(\mu, t)q - V^*q\|_{train} \\ & \leq N_{s_2}^{-1/2} \sup_{(\mu, t) \in \Theta \times \mathcal{T}} \|V(\mu, t)\|_\infty \|\hat{q} - q\|_{train} + \|V(\mu, t) - V^*\|_\infty \|q\|_{train} \\ & = N_{s_2}^{-1/2} \sup_{(\mu, t) \in \Theta \times \mathcal{T}} \|V(\mu, t)\|_\infty \|V(\mu, t)^T u - (V^*)^T u\|_{train} + \|V(\mu, t) - V^*\|_\infty \|(V^*)^T u\|_{train} \\ & \leq N_{s_2}^{-1/2} \sup_{(\mu, t) \in \Theta \times \mathcal{T}} \|V(\mu, t)\|_\infty \|V(\mu, t) - V^*\|_\infty \|u\|_{train} + \|V(\mu, t) - V^*\|_\infty \|V^*\|_\infty \|u\|_{train} \\ & \leq N_{s_2}^{-1/2} \sup_{(\mu, t) \in \Theta \times \mathcal{T}} \|u\|_{train} (\|V(\mu, t) - V^*\|_\infty (\|V(\mu, t)\|_\infty + 1)) \\ & \leq 2M \sup_{(\mu, t) \in \Theta \times \mathcal{T}} \|V(\mu, t) - V^*\|_\infty, \end{aligned}$$

where we used $\|u\|_{train} \leq |\sum_{j=1}^{N_{s_2}} M^2|^{1/2} = N_{s_2}^{1/2} M$ and additionally $\|V(\mu, t)\|_\infty = \|V^*\|_\infty = 1$ by orthonormality in the last arguments.

Do the Yarotski argument with the appropriate number of layers and weights.

□

4.3 Complexity Bounds for POD-DL-ROMs

Include discussed results with proof of Brivio et al.

Theorem 4.7. Let $\mathcal{G} : (\mu, \nu, t) \mapsto u(\mu, \nu, t)$ for any $(\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}$ be the parameter-to-solution map and let Assumption (2) and (3) be true. Let $1 > \delta > 0$ and $\varepsilon > 0$. Now assume we can collect freely a sufficient amount of training data $N_{data} = N_{data}(\delta, \varepsilon)$ and put it into the correlation matrix K as given in Definition (2.10) with σ_k^2 being its sorted eigenvalues. Now choose

$$N = \operatorname{argmin} \left\{ j \in \mathbb{N} \mid \sum_{k>j} \sigma_k^2 \leq \frac{m^2}{9} \varepsilon^2 \right\}.$$

Given the POD matrix $V \in \mathbb{R}^{N_h \times N}$ we now define the parameter-to-POD-coefficient map $\mathcal{Q} : (\mu, \nu, t) \mapsto q(\mu, \nu, t) := V^T u(\mu, \nu, t)$ for any $(\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}$. We assume that there exists $n > 0, \psi_* : \mathbb{R}^n \rightarrow \mathbb{R}^N, \psi'_* : \mathbb{R}^N \rightarrow \mathbb{R}^n$ that are respectively s -times and s' -times differentiable (with $s \gg s' \geq 2$), such that they enjoy the perfect embedding Assumption (4),

i.e.

$$\psi_*(\psi'_*(q(\mu, \nu, t))) = q(\mu, \nu, t), \quad \forall (\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}.$$

We further let

$$C_1 := \sup_{|\alpha| \leq s'} \sup_{v \in \mathbb{R}^N} |D^\alpha \psi'_*(v)|, \quad C_2 := \sup_{|\alpha| \leq s} \sup_{w \in \mathbb{R}^n} |D^\alpha \psi'_*(w)|.$$

Then there is a constant $c = c(\Theta, \Theta', \mathcal{T}, L, C_1, C_2, p, q, n, s, s')$ and a POD-DL-ROM architecture $V\hat{q} = V\psi \circ \phi : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^{N_h}$ composed of a decoder $\psi : \mathbb{R}^n \rightarrow \mathbb{R}^N$ having at most:

- $L_{n \rightarrow N} = c \log(\varepsilon^{-1})$ layers,
- $\omega_{n \rightarrow N} = cN\varepsilon^{-n/(s-1)} \log(\varepsilon^{-1})$ active weights,

and a reduced feed forward neural network $\phi : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^n$ having at most:

- $L_{(p+q+1) \rightarrow n} = c \log(\varepsilon^{-1})$ layers,
- $\omega_{(p+q+1) \rightarrow n} = cn\varepsilon^{-(p+q+1)} \log(\varepsilon^{-1})$ active weights,

such that $\mathbb{P}[\mathcal{E}_R < \varepsilon] > 1 - \delta$.

Proof. We can directly bound $\mathcal{E}_S = \mathcal{E}_S(N_{s_1}, N_{s_2}, N_t)$ independently of N , under the Assumption (2) by the Weak Law of Large Numbers, i.e. for all $1 > \delta > 0$ and $\varepsilon > 0$ there are N_{s_1}, N_{s_2}, N_t , such that

$$\mathbb{P}[\mathcal{E}_S((N_{s_1}, N_{s_2}, N_t)) < \varepsilon/3] > 1 - \delta. \quad (4.1)$$

This result provides the "probability" part of the PAC (probably almost correct) bound, which we want to show. Bounding $\mathcal{E}_{\text{NN}}$ and $\mathcal{E}_{\text{POD}}$ by $\varepsilon/3$ respectively will then provide the wanted result. For this, by choosing N as in the assumption of the statement, we simply rewrite

$$\mathcal{E}_{\text{POD}} := m^{-1} \sqrt{\sum_{k > N} \sigma_k^2} \leq \frac{\varepsilon}{3}. \quad (4.2)$$

To now bound the neural network approximation error, we will need to simply rewrite the error in terms of only the norm of the network itself;

$$\begin{aligned} \mathcal{E}_{\text{NN}} &= \left(\int_{\Theta \times \Theta' \times \mathcal{T}} \frac{\|Vq(\mu, \nu, t) - V\hat{q}(\mu, \nu, t)\|^2}{\|u(\mu, \nu, t)\|^2} d(\mu, \nu, t) \right)^{1/2} \\ &\leq m^{-1} \left(\int_{\Theta \times \Theta' \times \mathcal{T}} \|Vq(\mu, \nu, t) - V\hat{q}(\mu, \nu, t)\|^2 d(\mu, \nu, t) \right)^{1/2} \\ &\leq m^{-1} \left(|\Theta \times \Theta' \times \mathcal{T}| \sup_{(\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}} \|Vq(\mu, \nu, t) - V\hat{q}(\mu, \nu, t)\|^2 \right)^{1/2} \\ &\leq m^{-1} \left(|\Theta \times \Theta' \times \mathcal{T}| \|V\|_\infty^2 \sup_{(\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}} \|q(\mu, \nu, t) - \hat{q}(\mu, \nu, t)\|^2 \right)^{1/2} \\ &= m^{-1} |\Theta \times \Theta' \times \mathcal{T}|^{1/2} \sup_{(\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}} \|q(\mu, \nu, t) - \hat{q}(\mu, \nu, t)\|. \end{aligned} \quad (4.3)$$

whereas we split the latter error into both parts, since $\hat{q} = \psi \circ \phi$ per definition. This means employing two types of error estimations in the matter of finding the respective optimal maps for

the feed forward estimation of the latent dynamics via $\phi_* : \mathbb{R}^{(p+q+1)} \rightarrow \mathbb{R}^n$ and the decoder lift via $\psi_* : \mathbb{R}^n \rightarrow \mathbb{R}^N$.

- Let $\mathcal{S}_N := \{q = \mathcal{Q}(\mu, \nu, t) \mid (\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}\}$, then the image of the optimal embedding $\mathcal{V}_N = \psi'_*(\mathcal{S}_N)$ is such that $\text{diam}(\mathcal{V}_N) \leq LC_1 \text{diam}(\Theta \times \Theta' \times \mathcal{T})$, given the Assumption (3) by continuity.

Make this rigorous

Thus, by Theorem Gühring, there is a ReLU Neural Network $\psi : \mathbb{R}^n \rightarrow \mathbb{R}^N$, such that

$$\sup_{v \in \mathcal{V}_N} \|\psi(v) - \psi_*(v)\| < \frac{m}{6} |\Theta \times \Theta' \times \mathcal{T}|^{-1/2} \varepsilon, \quad (4.4)$$

$$\text{ess sup}_{v, v' \in \mathcal{V}_N} \frac{|(\psi - \psi_*)(v) - (\psi - \psi_*)(v')|}{|v - v'|} < \frac{m}{6} |\Theta \times \Theta' \times \mathcal{T}|^{-1/2} \varepsilon, \quad (4.5)$$

with $L_{n \rightarrow N} = c \log(\varepsilon^{-1})$ layers and $\omega_{n \rightarrow N} = cN\varepsilon^{-n/(s-1)} \log(\varepsilon^{-1})$ active weights.

Make this rigorous

- Now let $\phi_*(\mu, \nu, t) = \psi'_*(q(\mu, \nu, t))$ for $(\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}$, which is Lipschitz continuous, with constant LC_1 , and thus, by

Make this rigorous

Theorem Yarotski, there exists a ReLU Neural Network $\phi : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^n$, such that

$$\sup_{(\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}} \|\phi(\mu, \nu, t) - \phi_*(\mu, \nu, t)\| < \frac{m}{6C_3} |\Theta \times \Theta' \times \mathcal{T}|^{-1/2} \varepsilon, \quad (4.6)$$

with $L_{(p+q+1) \rightarrow n} = c \log(\varepsilon^{-1})$ layers and $\omega_{(p+q+1) \rightarrow n} = cn\varepsilon^{-(p+q+1)} \log(\varepsilon^{-1})$ active weights.

Starting from the last inequality in (4.3) we can perform the split in a clean fashion using the triangle inequality and reminding ourselves of the Assumption (4), which guarantees $q = \psi_*(\psi'_*)$, and thus gives with both neural network estimates in Equations (4.4) and (4.6)

$$\begin{aligned} & \sup_{(\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}} \|q(\mu, \nu, t) - \hat{q}(\mu, \nu, t)\| \\ & \leq \sup_{(\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}} (\|\psi_*(\psi'_*(\mu, \nu, t)) - \psi(\phi_*(\mu, \nu, t))\| + \|\psi(\phi_*(\mu, \nu, t)) - \psi(\phi(\mu, \nu, t))\|) \\ & \leq \sup_{v \in \mathcal{V}_N} \|\psi_*(v) - \psi(v)\| + C_3 \sup_{(\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}} (\|\psi(\phi_*(\mu, \nu, t)) - \psi(\phi(\mu, \nu, t))\|) \\ & < \frac{m}{6} |\Theta \times \Theta' \times \mathcal{T}|^{-1/2} \varepsilon + C_3 \frac{m}{6C_3} |\Theta \times \Theta' \times \mathcal{T}|^{-1/2} \varepsilon = \frac{\varepsilon}{3} m |\Theta \times \Theta' \times \mathcal{T}|^{-1/2}, \end{aligned}$$

which cancels with $m^{-1} |\Theta \times \Theta' \times \mathcal{T}|^{1/2}$ to get

$$\mathcal{E}_{\text{NN}} < \frac{\varepsilon}{3}. \quad (4.7)$$

Now putting together all three estimates (4.1), (4.2) and (4.7) with the reasoning of Theorem (4.2) finally gives

$$\mathcal{E}_R \leq \mathcal{E}_S + \mathcal{E}_{\text{POD}} + \mathcal{E}_{\text{NN}} < \frac{\varepsilon}{3} + \frac{\varepsilon}{3} + \frac{\varepsilon}{3} = \varepsilon,$$

with higher probability than $1 - \delta$. □

4.4 Complexity Bounds for DOD-DL-ROMs

Extrapolate to DOD-DL-ROM

Theorem 4.8. Let $\mathcal{G} : (\mu, \nu, t) \mapsto u(\mu, \nu, t)$ for any $(\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}$ be the parameter-to-solution map and let Assumption (2) and (3) be true. Let $1 > \delta > 0$ and $\varepsilon > 0$. Now assume we can collect freely a sufficient amount of training data $\Theta_{\text{data}}, \Theta'_{\text{data}}$ and $\mathcal{T}_{\text{data}}$ of size $N_{\text{data}} = N_{\text{data}}(\delta, \varepsilon)$ and put all samples of $u(\mu, \nu, t)_{j=1}^{N_{s_2}}$ into a correlation matrix $K_{\mu, t}$ for each sample $(\mu, t) \in \Theta_{\text{data}} \times \mathcal{T}_{\text{data}}$ as given in Definition (2.10) with $\sigma_k(\mu, t)_k^2$ being its sorted eigenvalues. Now choose

$$N_{\text{DOD}} = \operatorname{argmin} \left\{ j \in \mathbb{N} \mid \sup_{(\mu, t) \in \Theta_{\text{data}} \times \mathcal{T}_{\text{data}}} \sum_{k > j} \sigma(\mu, t)_k^2 \leq \frac{m^2}{16} \varepsilon^2 \right\}.$$

For each pair $(\mu, t) \in \Theta_{\text{data}} \times \mathcal{T}_{\text{data}}$ given the respective POD matrix, so the best possible projection given the data set (refer to (2.12)), $V^*(\mu, t) \in \mathbb{R}^{N_h \times N_{\text{DOD}}}$ we now define the parameter-to-optimal-coefficient map $\mathcal{Q} : (\mu, \nu, t) \mapsto q(\mu, \nu, t) := V^*(\mu, t)^T u(\mu, \nu, t)$ for any $(\mu, \nu, t) \in \Theta_{\text{data}} \times \Theta' \times \mathcal{T}_{\text{data}}$. We assume that there exists $n > 0$, $\Psi_* : \mathbb{R}^n \rightarrow \mathbb{R}^{N_{\text{DOD}}}$, $\Psi'_* : \mathbb{R}^{N_{\text{DOD}}} \rightarrow \mathbb{R}^n$ that are respectively s -times and s' -times differentiable (with $s > s' \geq 2$), such that they enjoy the perfect embedding Assumption (4), i.e.

$$\Psi_*(\Psi'_*(q(\mu, \nu, t))) = q(\mu, \nu, t), \quad \forall (\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}.$$

We further let

$$C'_1 := \sup_{|\alpha| \leq s'} \sup_{v \in \mathbb{R}^{N_{\text{DOD}}}} |D^\alpha \Psi'_*(v)|, \quad C'_2 := \sup_{|\alpha| \leq s} \sup_{w \in \mathbb{R}^n} |D^\alpha \Psi'_*(w)|.$$

Then there is a constant $c = c(\Theta, \Theta', \mathcal{T}, L, C_1, C_2, p, q, n, s, s')$ and a DOD-DL-ROM architecture $V\hat{q} = V_{\text{DOD}}\Psi \circ \phi : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^{N_h}$ composed of a decoder $\Psi : \mathbb{R}^n \rightarrow \mathbb{R}^{N_{\text{DOD}}}$ having at most:

- $L_{n \rightarrow N_{\text{DOD}}} = c \log(\varepsilon^{-1})$ layers,
- $\omega_{n \rightarrow N_{\text{DOD}}} = c N_{\text{DOD}} \varepsilon^{-n/(s-1)} \log(\varepsilon^{-1})$ active weights,

a reduced feed forward neural network $\phi : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^n$ having at most:

- $L_{(p+q+1) \rightarrow n} = c \log(\varepsilon^{-1})$ layers,
- $\omega_{(p+q+1) \rightarrow n} = cn \varepsilon^{-(p+q+1)} \log(\varepsilon^{-1})$ active weights,

and a DOD Module $V_{\text{DOD}} : \mathbb{R}^{p+1} \rightarrow \mathbb{R}^{N_h \times N_{\text{DOD}}}$ having at most:

- $L_{(p+1) \rightarrow N_h \times N_{\text{DOD}}} = c \log(\varepsilon^{-1})$ layers,
- $\omega_{(p+1) \rightarrow N_h \times N_{\text{DOD}}} = c N_h N_{\text{DOD}} \varepsilon^{-(p+1)} \log(\varepsilon^{-1})$ active weights,

such that $\mathbb{P}[\mathcal{E}_R < \varepsilon] > 1 - \delta$.

Proof. Similar as to the proof of Theorem (4.7), we will try to bound each quantity of the statement in Theorem (4.5), but with the catch, that $\mathcal{E}_{\text{DOD}}$ is stochastic in nature, and thus we will employ

Lemma (4.6) to find an auxiliary term, which is independent of the data, to which the approximation of the neural net of the DOD is sufficiently "close".

The sampling error can be bounded in the same way as before, i.e. we can directly bound $\mathcal{E}_S = \mathcal{E}_S(N_{s_1}, N_{s_2}, N_t)$ independently of N_{DOD} , under the Assumption (2) by the Weak Law of Large Numbers, i.e. for all $1 > \delta > 0$ and $\varepsilon > 0$ there are N_{s_1}, N_{s_2}, N_t , such that

$$\mathbb{P}[\mathcal{E}_S((N_{s_1}, N_{s_2}, N_t)) < \varepsilon/4] > 1 - \delta. \quad (4.1)$$

Now recall the definition of $\mathcal{E}_{\text{DOD}}$ in the proof of Theorem (4.5) and further establish

$$\begin{aligned} \mathcal{E}_{\text{DOD}} &= \frac{m^{-1}|\Theta \times \Theta' \times \mathcal{T}|}{N_{\text{data}}^{1/2}} \left| \sum_{i=1}^{N_{s_1}} \sum_{j=1}^{N_{s_2}} \sum_{k=1}^{N_t} \|u(\mu_i, \nu_j, t_k) - V(\mu_i, t_k)V(\mu_i, t_k)^T u(\mu_i, \nu_j, t_k)\|^2 \right|^{1/2} \\ &\leq m^{-1} \sup_{(\mu, t) \in \Theta \times \mathcal{T}} \left| \frac{|\Theta \times \Theta' \times \mathcal{T}|}{N_{s_2}} \sum_{j=1}^{N_{s_2}} \|u(\mu, \nu_j, t) - V(\mu, t)V(\mu, t)^T u(\mu, \nu_j, t)\|^2 \right|^{1/2}, \end{aligned}$$

enabling us to use Proposition (2.12) in order to switch to the eigenvalues $\sigma(\mu, t)_k$ for $(\mu, t) \in \Theta_{\text{data}} \times \mathcal{T}_{\text{data}}$ of the theoretical POD matrix $V^*(\mu, t)$ by employing the Lemma (4.6) with the bound

$$\frac{\varepsilon m^2}{4|\Theta \times \Theta' \times \mathcal{T}|^{1/2}},$$

which directly gives us the approximation result via the definition of N_{DOD}

$$\mathcal{E}_{\text{DOD}} < \frac{\varepsilon}{4} + m^{-1} \sup_{(\mu, t) \in (\Theta_{\text{data}} \times \mathcal{T}_{\text{data}})} \sqrt{\sum_{k > N_{\text{DOD}}} \sigma(\mu, t)_k^2} \leq \frac{\varepsilon}{2}. \quad (4.2)$$

Include the cost via Yarotski

Argue the actual part for the NN error

Moreover we can use the exact same arguments as in the bound for $\mathcal{E}_{\text{NN}}$ for the POD-DL-ROM as in the proof of Theorem (4.7), which yields

$$\mathcal{E}_{\text{NN}} < \frac{\varepsilon}{4}. \quad (4.3)$$

Finally putting each Equation, i.e. (4.1), (4.2) and (4.3), together, we get

$$\mathcal{E}_R \leq \mathcal{E}_S + \mathcal{E}_{\text{DOD}} + \mathcal{E}_{\text{NN}} < \frac{\varepsilon}{4} + \frac{\varepsilon}{2} + \frac{\varepsilon}{4} = \varepsilon,$$

with probability greater than $1 - \delta$ concluding our proof. $\square$

4.5 Comparison between existing Techniques

Chapter 5

Numerical Experiments

Include all numerical experiments

Find increase of performance depending on latent dimension

Give benchmark on pass forward times

Discuss comparison between different DL-ROMs

Chapter 6

Conclusion

Discuss results

Insert outlook for future ideas and implementation

Bibliography

- Alt, H. W. (2016, July). *Linear functional analysis*. Springer London. <https://doi.org/10.1007/978-1-4471-7280-2>
- Bauer, F. L., & Fike, C. T. (1960). Norms and exclusion theorems. *Numerische Mathematik*, 2, 137–141. <https://api.semanticscholar.org/CorpusID:121278235>
- Bramble, J. H., & Hilbert, S. R. (1970). Bounds for a class of linear functionals with applications to Hermite interpolation. *Numerische Mathematik*, 16(4), 362–369. <https://doi.org/10.1007/BF02165086>
- Brezis, H. (2011). *Functional analysis, sobolev spaces and partial differential equations* (1st ed.). Springer New York. <https://doi.org/10.1007/978-0-387-70914-7>
- Brezis, H., & Browder, F. (1998). Partial differential equations in the 20th century. *Advances in Mathematics*, 135(1), 76–144. <https://doi.org/https://doi.org/10.1006/aima.1997.1713>
- Dacorogna, B. (2004, November). *Introduction to the calculus of variations*. <https://doi.org/10.1142/p361>
- Franco, N. R., Manzoni, A., Zunino, P., & Hesthaven, J. S. (2024). Deep orthogonal decomposition: A continuously adaptive data-driven approach to model order reduction [arXiv:2404.18841]. <https://arxiv.org/abs/2404.18841>
- Fresca, S., Dede, L., & Manzoni, A. (2020). A comprehensive deep learning-based approach to reduced order modeling of nonlinear time-dependent parametrized pdes. <https://arxiv.org/abs/2001.04001>
- Grepl, M. A., & Patera, A. T. (2005). A posteriori error bounds for reduced-basis approximations of parametrized parabolic partial differential equations. *ESAIM: Mathematical Modelling and Numerical Analysis*, 39, 157–181. <https://doi.org/10.1051/m2an:2005006>
This article establishes rigorous and computable a posteriori error bounds for reduced-basis approximations of parametrized parabolic PDEs. The authors develop an error certification framework based on energy estimates and residual evaluations, ensuring both efficiency and reliability in model reduction. Their methodology combines reduced-basis projection with an offline/online decomposition strategy, making the approach suitable for many-query and real-time simulation contexts. The work is foundational for certified model order reduction, especially in time-dependent problems.
- Gühring, I., Kutyniok, G., & Petersen, P. (2019). Error bounds for approximations with deep relu neural networks in $W^{s,p}$ norms. <https://arxiv.org/abs/1902.07896>
- Haasdonk, B., & Ohlberger, M. (2008). Reduced basis method for finite volume approximations of parametrized linear evolution equations. *ESAIM: Mathematical Modelling and Numerical Analysis*, 42, 277–302. <https://doi.org/10.1051/m2an:2008001>
- Hesthaven, J., Pagliantini, C., & Rozza, G. (2022). Reduced basis methods for time-dependent problems. *Acta Numerica*, 31, 265–345. <https://doi.org/10.1017/S0962492922000058>
- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2), 251–257. [https://doi.org/https://doi.org/10.1016/0893-6080\(91\)90009-T](https://doi.org/https://doi.org/10.1016/0893-6080(91)90009-T)

- Quarteroni, A., Manzoni, A., & Negri, F. (2015). *Reduced basis methods for partial differential equations*. Springer. <https://doi.org/10.1007/978-3-319-15431-2>
- Siena, P., Girfoglio, M., & Rozza, G. (2022). An introduction to pod-greedy-galerkin reduced basis method. <https://arxiv.org/abs/2203.08532>
- Yarotsky, D. (2017). Error bounds for approximations with deep relu networks. *Neural Networks*, 94, 103–114. <https://doi.org/https://doi.org/10.1016/j.neunet.2017.07.002>